

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«АСТРАХАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ ИМ. В.Н.
ТАТИЩЕВА»**

Факультет цифровых технологий и кибербезопасности

Кафедра информационных технологий

КУРСОВАЯ РАБОТА

**Разработка информационной системы для анализа эффективности
микросервисной архитектуры**

выполнена в рамках изучения дисциплины

«Междисциплинарный проект»

Направление подготовки: 09.04.02 Информационные системы и технологии.

Направленность (профиль): «Прикладные информационные технологии»

Исполнитель: студент группы ДИФ-15
Кузургалиев Р.А. _____

Научный руководитель: к.п.н., доцент
кафедры ИТ
Кириллова Т.В. _____

Инженер 2 категории ГрКБОККИИ ОИБ
УКЗ ООО «Газпром добыча
Астрахань»
Лазарев Н.В. _____

Оценка _____

«20» апреля 2026г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
2 ОБЗОР ИЗВЕСТНЫХ МЕТОДОВ И СРЕДСТВ РЕШЕНИЯ ПРОБЛЕМЫ...	4
2.1. Краткое описание предметной области	4
2.2. Анализ релевантных стандартов по теме исследования.	12
2.3. Сравнение и оценка методологических, технологических, алгоритмических и программных решений по теме исследования	17
2.3.1. Программные решения задачи балансировки и распределения	17
2.3.2. Алгоритмы балансировки и распределения	21
2.3.3. Основные положения нагрузочного тестирования	27
2.3.4. Инструменты нагрузочного тестирования	32
2.4. Постановка задачи исследования	34
3 ОБЗОР ИЗВЕСТНЫХ МЕТОДОВ И СРЕДСТВ РЕШЕНИЯ ПРОБЛЕМЫ.	36
3.1. Основные положения методики	36
3.2. Основные принципы нагрузочного тестирования	37
3.3. Использование памяти.....	39
3.4. Использование ресурсов процессора	41
3.5. Энергопотребление	44
3.6. Стабильность сети	47
3.7. Производительность	49
3.8. Характеристика брокера сообщений	52
3.9. Характеристика ошибок и исключений	54
3.10. Общие взаимосвязи	56
ЗАКЛЮЧЕНИЕ	60
СПИСОК ЛИТЕРАТУРЫ.....	61

ЗАДАНИЕ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«АСТРАХАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ ИМЕНИ В. Н.
ТАТИЩЕВА»

Направление «Информационные системы и технологии»
Кафедра информационных технологий

«14» февраля 2026 г.

З А Д А Н И Е

на курсовую работу студентки
Кузургалиев Радмир Алексеевич

- 1 Тема курсовой работы: «Разработка информационной системы для анализа эффективности микросервисной архитектуры»
 - 2 Проанализировать теоретический материал в области микросервисной и монолитной архитектуры;
 - 3 Проанализировать процесс балансировки в микросервисной архитектуре клиент-серверного приложения
 - 4 Составить алгоритм, позволяющий реализовать выбор архитектурного паттерна для высоконагруженных информационных систем
- Задание принял к исполнению Кузургалиев Радмир Алексеевич

ВВЕДЕНИЕ

Целью курсовой работы является изучение современных архитектурных подходов, методологии проектирования и критериев оценки эффективности высоконагруженных информационных систем в рамках области туризма.

Проведено исследование в области повышения эффективности информационных систем в сфере туризма через выбор и оптимизацию существующих архитектурных решений. Был проведён сравнительный анализ монолитной, сервис-ориентированной, микросервисной и бессерверной архитектур. Были оценены основные методы и практики реализации микросервисной архитектуры, а также технологические стеки для реализации и тестирования информационной системы. Работа фокусируется на анализе свойств микросервисного паттерна (отказоустойчивость, производительность, масштабируемость) и разработке методики для проектирования высоконагруженных систем. Определена проблема отсутствия комплексной оценки архитектуры и алгоритма балансировки микросервисов.

Исследование связано с реализацией национального проекта «Туризм и гостеприимство», в рамках которого были определены цели по увеличению внутреннего и въездного туристского потока.

В серии стандартов ISO/IEC, ГОСТ и RFC были рассмотрены практики, касающиеся надёжности, доступности, производительности и обслуживаемости IT-систем и протоколы передачи информации.

На основе анализа формулируется задача распределения/оптимизации: как распределить ресурсы системы (серверные мощности) на микросервисы таким образом, чтобы обеспечить максимальную производительность и отказоустойчивость системы.

Итоговым практическим результатом станет макет автореферата магистерской диссертации, отражающая все ключевые элементы исследования, а также научная статья.

2 ОБЗОР ИЗВЕСТНЫХ МЕТОДОВ И СРЕДСТВ РЕШЕНИЯ ПРОБЛЕМЫ

2.1. Краткое описание предметной области

Перечень критических технологий Российской Федерации - один из основных инструментов государственной политики Российской Федерации в области развития отечественной науки и технологий. Его формирование предусмотрено «Основами политики Российской Федерации в области развития науки и технологий на период до 2010 года и дальнейшую перспективу», утверждёнными Указом Президента Российской Федерации от 30 марта 2002 года № Пр-576[10].

Перечень критических технологий Российской Федерации утверждается в соответствии с поручением Президента Российской Федерации от 21 мая 2006 года № Пр-842[11] о корректировке приоритетных направлений развития науки, технологий и техники в Российской Федерации и перечня критических технологий Российской Федерации решениями Президента по представлению Правительства не реже одного раза в четыре года. Одновременно утверждаются Приоритетные направления развития науки, технологий и техники в Российской Федерации.

Так Указом Президента РФ от 7 июля 2011 года № 899[12] был утверждён перечень критических технологий Российской Федерации, в рамках которого были выделено направление «Технологии и программное обеспечение распределённых и высокопроизводительных вычислительных систем».

Другим актуализирующим фактором являются национальные проекты. Национальные проекты — один из основных инструментов достижения утвержденных Президентом национальных целей развития и реализации программы социально-экономического развития России до 2030 года. Они содержат ключевые решения, направленные на укрепление экономики страны, обеспечение технологического суверенитета и улучшение жизни граждан.

В Национальном проекте «Туризм и гостеприимство» планируется повышение роли туристической отрасли в экономике страны, путём увеличения общего количества туристского потока. В рамках данной федеральной программы определено развитие и обеспечение функционирования информационных систем, направленных на реализацию государственных услуг и функций в сфере туризма. На рисунке 2.1 изображена динамика туристского потока за последние годы.



Рисунок 2.1 – Динамика туристского потока РФ

Туристическая отрасль отличается большим объемом обрабатываемой и хранимой информации. Современные информационные системы, которые предоставляют пользователю туристические услуги вынуждены обрабатывать сотни тысяч запросов в секунду [13], что обуславливает необходимость быстродействия и распределённое хранение данных приложения.

Архитектура программного обеспечения претерпела существенную эволюцию за последние десятилетия. Начиная с крупных монолитных приложений на мейнфреймах и заканчивая современными распределенными микросервисами, каждый этап развития решал насущные проблемы своего времени – и зачастую порождал новые. Уже начиная с 60-ых годов 20-го века монолитная архитектура доминирует в мире как основной способ реализации программного обеспечения. На заре компьютерной эры ещё не существовало никаких сетевых технологий, а аппаратная часть была очень дорогой и ещё не настолько мощной, чтобы поддерживать множество обрабатываемых процессов, поэтому в этих условиях концепция единого приложения получила наибольшую популярность и отлично вписывалась в контекст тех лет.

Суть монолитного подхода заключается в том, что разработанное приложение представлено как единое целое. Весь функционал объединён в единой кодовой базе и процессе выполнения. Развертывание приложения с такой архитектурой происходит целиком: обновление и масштабирование происходит только в рамках всего блока.

В контексте своего времени преимущества монолитов заключаются в следующем:

- Один исполняемый файл содержит сразу всё приложения, что обуславливает согласованность версий компонентов.
- Как правило монолит использует единую базу данных для всего приложения, а транзакции обеспечивают целостность данных.
- Вызов между модулями происходит в рамках одного процесса, отсутствуют сетевая задержка и сериализация данных.
- В рамках проведения тестирования достаточно развернуть один экземпляр приложения.

По мере развития индустрии монолиты начали сталкиваться с серьезными проблемами. Главной головоломкой для разработчиков стала сложность масштабируемости и расширяемости приложений: в крупных монолитных системах любые изменения требовали тщательного регрессионного тестирования всего приложения, отдельные команды мешали друг другу, разрабатывая в одной кодовой базе.

Однако уже к концу 20-го века крупным компаниям стало что огромные связанные монолиты неудовлетворительно справляются с возросшей сложностью корпоративных систем. Требовался новый подход к архитектуре, позволяющий разбивать системы на более простые, управляемые части.

В это же время индустрия пыталась привести порядок внутри монолита, структурируя компоненты системы, таким образом появились многослойные архитектуры и деление приложения на модули, в 90-е годы начала приобретать популярность трёхслойная модель: «интерфейс – бизнес-логика - данные». По

сути данная модель всё ещё продолжала оставаться монолитной, но ответственность была разделена между компонентами.

В рамках данной концепции стала очевидной проблема чрезмерной связности модулей системы - часто множество классов превращались в «лапшу» из зависимостей. Также крайне проблематичным стал процесс развёртывания: для выпуска новой версии приходилось выполнять регрессионные тесты на всем приложении.

Одновременно практики объектно-ориентированного проектирования и компонентного подхода способствовали созданию более модульных монолитов. Разработчики стали разбивать свой код на пакеты, модули и библиотеки по функциональным областям. Так появлялись отдельные модули внутри единого приложения. Подобная архитектура получила название модульный монолит – приложение всё ещё оставалось единым, но с независимыми модулями, имеющими чёткие границы.

К началу нового тысячелетия наиболее крупные компании по-прежнему работали с монолитными системами, в то же время пытаясь сделать их максимально модульными и слоистыми. Подобная практика стала фундаментом будущих сервисных архитектур.

С появлением и широким распространением персональных компьютеров по всему миру происходит повсеместное распространение веб-сервисов. К середине 00-ых годов 21-го века наметилась общая концепция сервисно-ориентированной архитектуры (SOA), ключевой идеей которой является повторное использование отдельных компонентов (сервисов) и интероперабельность. Несмотря на имеющиеся минусы SOA подготовила почву для следующего шага – в индустрии стала очевидна тенденция отделения сервисов по бизнес-границам.

Сервисно-ориентированная архитектура разбивает систему на более крупные службы (services), которые представляют определенные бизнес-функции и взаимодействуют через четко определенные интерфейсы. В отличие от модулей внутри монолита, сервисы в SOA часто развертываются отдельно,

могут работать на разных серверах, а общение идет по сети (через SOAP/HTTP или сообщений через шину). На рисунке 2.2 представлена схема работы SOA приложения.

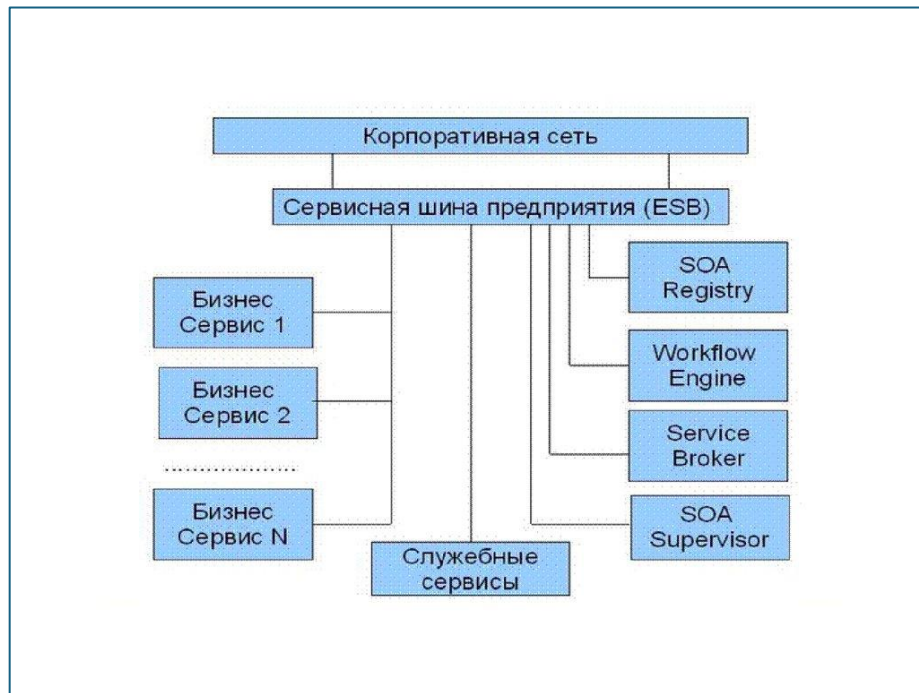


Рисунок 2.2 – Схема работы SOA приложения

Данный подход решал сразу несколько проблем:

- Появлялась возможность выделения коды в отдельные сервисы, которые вызывались в разные места кодовой базы, вместо привычного дублирования логики.
- Благодаря появлению единых стандартов стало возможным взаимодействовать между сервисами, что особенно понравилось крупным корпорациям.
- В рамках SOA команды начали работать более независимо друг от друга, у каждого сервиса – свой цикл релизов.

Несмотря на решение явных, давно преследовавших разработчиков проблем, SOA столкнулась с новыми сложностями. Зачастую сервисы в SOA часто были крупными по функциональности и использовали тяжелые протоколы для обмена информацией и оркестрации – всё это добавляло сложность и накладные расходы. Системы стали распределенными,

но не упростились – наоборот, возник слой сложной инфраструктуры: регистры сервисов, брокеры сообщений, оркестровщики. Центральная шина сообщений стала узким местом в информационных системах и точкой отказов, добавление нового сервиса требовало настройки множества соглашений на шине. Тем не менее, SOA приучила индустрию мыслить сервисными контрактами, отделять сервисы по бизнес-границам и решать проблемы распределенных систем.

К началу 10-ых годов на фоне удешевления производства компьютеров и смартфонов происходит заметный рост пользователей сети Интернет, общая нагрузка на информационные системы крупных компаний становится очевидной – многие старые приложения перестают эффективно работать, а иногда даже и вовсе прекращают свою деятельность. На этом фоне решением данной проблемы стало продолжение идей 00-ых годов - дальнейшая декомпозиция сервисной архитектуры. Микросервисная архитектура является логическим завершением пути дробления ответственности за работу приложения между её компонентами. Данный подход стали популярны одновременно с развитием облачных технологий, контейнеризации и DevOps-культуры.

Микросервисы - это дальнейшее дробление сервисно-ориентированной архитектуры на более мелкие, автономно развертываемые компоненты. Каждая такая служба выполняет четко очерченную функцию, работает в собственном процессе, может иметь собственную базу данных, и общается с другими по легким протоколам (обычно REST/HTTP или message queue). В идеале микросервис можно разработать, тестировать, задеплоить и масштабировать независимо от остальных. Одновременно с развитием микросервисной архитектуры стала развиваться DevOps-культура. На рисунке 2.3 представлена сравнительная диаграмма работы монолитного и микросервисного приложения.

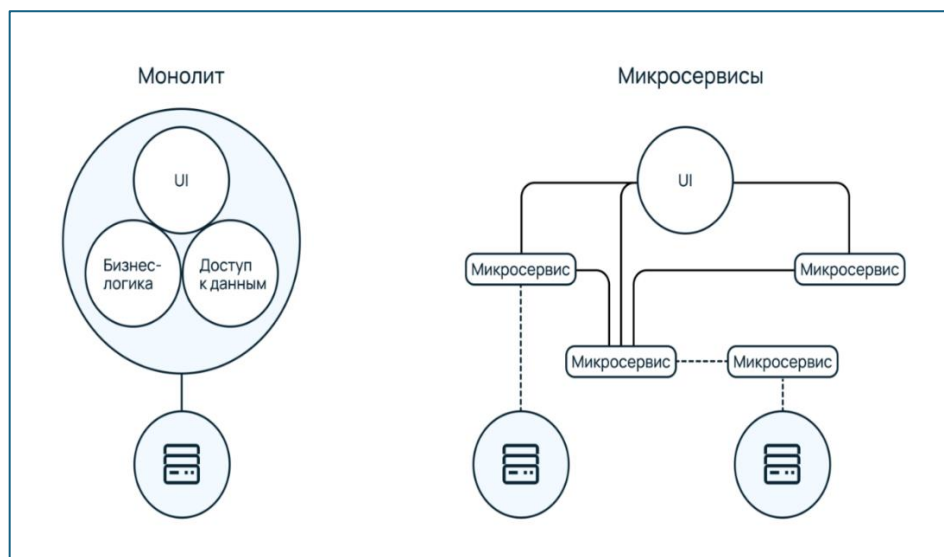


Рисунок 2.3 – Сравнительная диаграмма работы монолитного и микросервисного приложения.

У микросервисной архитектуры есть следующие преимущества:

- С точки зрения ресурсов процесс масштабирования стал гораздо эффективнее: каждая служба по мере нагрузки на её функционал могла масштабироваться независимо от других компонентов приложения.
- У команд разработчиков ускорился цикл разработки: Небольшие кодовые базы проще менять, протестировать и выкатить в продакшен.
- Благодаря данному подходу значительно увеличилась отказоустойчивость программных продуктов, так как сбой одного микросервиса не разрушает работу всей системы.
- Увеличилось понимание кода: каждый микросервис реализует относительно узкий контекст, то есть крайне ограниченную по функционалу кодовую базу.

Однако на практике выяснилось, что микросервисная архитектура приносит собственную сложность и проблемы.

- Сильно усложнилась инфраструктура приложений. Вместо одного приложения нужно **управлять множеством** маленьких сервисов. Встали задачи **сервис-менеджмента: регистрация/обнаружение**

сервисов, балансировка нагрузки, мониторинг и логирование распределенной системы.

- В связи с разбитием приложения на микросервисы, стала актуальной проблема сетевых задержек: каждый межсервисный запрос несет риск: сеть может тормозить или пакеты потеряются. Высоконагруженные системы начинают страдать от задержек из-за множества удаленных вызовов.
- В монолите все часто крутилось вокруг одной базы с транзакциями. В микросервисах, у каждого сервиса как правило своё хранилище, **глобальные транзакции** сложно реализуемы. Данные распределены, крайне затруднилось написание сложных шаблонов для процессов, затрагивающих несколько сервисов.
- Микросервисная архитектура требует совершенно другой культуры – **децентрализованной**. Команды разработчиков должны уметь принимать больше ответственности при создании, тестировании и развертывании приложений, к тому же для этого требуется высокий уровень автоматизации.

Таким образом, микросервисы решают проблему растущей сложности тем, что делают сложность явной и выносят ее наружу. С ростом системы рост сложности неизбежен, и микросервисы - это способ управлять ею после определенного порога.

Однако, к 2020-м годам пришло отрезвление: выяснилось, что микросервисы подходят не всем и не всегда. Внедрение микросервисной архитектуры без достаточных оснований приводит к чрезмерной сложности разработки.

Параллельно с этим в 2014 году компания Amazon предложила новую концепцию – «функции как сервис». Это вовсе не означает отсутствие серверов, а подразумевает, что разработчикам не нужно управлять серверами. Однако от данной технологии в 2023 году отказались сами разработчики, признав, что данный подход декомпозиции системы чересчур

избыточен. Данный шаг позволил сократить накладные расходы на 90% и устранить узкие места производительности. [14].

Код загружается в виде небольших функций, которые провайдер автоматически выполняет в ответ на события, масштабирует и останавливает, когда не нужно. Бессерверную архитектуру можно рассматривать как дальнейшее разбиение микросервисов на функции – экстремальный вариант, когда каждое независимое действие, например, обработчик события развертывается отдельно. Из преимуществ данного подхода можно выделить упрощение работы программиста: разработчик пишет только функцию, провайдер (AWS, Azure и т.д.) сам обеспечивает запуск на инфраструктуре, распределение нагрузки. К тому же удалось уменьшить затрачиваемое на внедрение нового функционала время: функции маленькие, независимые – их легко писать и развертывать по отдельности.

Минусы данной концепции логически вытекают из её плюсов. Реализованные функции часто зависят от конкретного облачного провайдера, перенос такого приложения на другую материальную базу сопровождается дополнительными трудностями. В связи с распределённой кодовой базой приложения традиционные инструменты отладки и тестирования плохо применимы. Нужно использовать облачные средства логирования, трейсинга, что добавляет иной раз загадок.

Бессерверная архитектура нашла свое применение в событиях IoT, мобильных приложений, обработке медиа, автоматизации и т.д. Однако он не вытеснил микросервисы, а скорее дополнил палитру архитектурных стилей. Многие системы сочетают подходы: где-то используют длительно работающие сервисы, а где-то – функции на вызов.

2.2. Анализ релевантных международных стандартов по теме исследования.

Прямых стандартов, описывающих требования к архитектуре информационных систем нет, однако есть несколько ключевых стандартов, которые создают фундамент и концептуальную основу. Одним из них является

ISO/IEC/IEEE 42010:2011, в Российской классификации ГОСТ Р 57100-2016 «Системная и программная инженерия. Описание архитектуры»[15]. Данный стандарт определяет способ, с помощью которого осуществляются организация и выражение описания архитектуры. В нём определены условия для реализации желаемых свойств структур архитектуры и языков описания архитектуры в порядке целесообразной поддержки разработки и использования описаний архитектуры. В стандарте содержатся основы для сравнения и объединения структур архитектуры и языков описания архитектуры путем обеспечения общей онтологии для определения их содержания. Данный документ используется для установления последовательной практики при разработке описаний архитектуры, ее структуры и языков описания архитектуры в пределах контекста жизненного цикла и его процессов, а также применяется для оценки соответствия описания архитектуры, структуры архитектуры, языка описания архитектуры или точки зрения на архитектуру.

Другим стандартом, который регламентирует оценку информационной системы как конечного продукта является ISO/IEC 25010:2011, в Российской классификации ГОСТ Р ИСО/МЭК 25010—2015 «Системная и программная инженерия. Требования и оценка качества систем и программного обеспечения. Модели качества систем и программного продукта»[16]. Данный стандарт позволяет оценить и обосновать выбор архитектуры с точки зрения качества исходя из следующих характеристик:

- Функциональная пригодность.
- Уровень производительности.
- Совместимость.
- Удобство использования.
- Надёжность.
- Защищённость.
- Сопровождение.
- Переносимость.

Исходя из вышеописанных стандартов, в рамках особенностей предметной области, а именно высокая средняя и пиковая нагрузка, большой объём хранимой информации, становится очевидным, что **микросервисная архитектура является наиболее предпочтительной для дальнейшего анализа.**

Обмен данными между компонентами информационной системы регламентируются международными стандартами RFC 9110, RFC 8259 и ISO/IEC 19464. Стандарт RFC 8259 определяет строгий синтаксис и грамматику формата JSON как к базовой единице обмена информации[17]. Он устанавливает требования к корректному формированию объектов, массивов, строк, чисел, логических значений и null, что обеспечивает гарантированную интерпретацию данных различными программными компонентами, написанными на любых языках программирования.

Стандарт RFC 9110 регламентирует существование протокола HTTP — это семейство протоколов прикладного уровня без поддержки состояния, использующих модель «запрос-отклик» и имеющих единый базовый интерфейс, расширяемую семантику и самоописывающиеся сообщения для обеспечения гибкого взаимодействия с сетевыми информационными системами на основе гипертекста[18]. HTTP скрывает детали реализации сервиса, предоставляя клиентам унифицированный интерфейс, не зависящий от типа представляемых ресурсов. Серверам не нужно знать о целях каждого клиента, запросы могут рассматриваться изолированно без привязки к определённому типу клиента или предопределённой последовательности действий приложения. Это позволяет эффективно применять реализации общего назначения в разном контексте, упрощает взаимодействия и обеспечивает независимое развитие с течением времени. HTTP также предназначен для использования в качестве промежуточного протокола (посредника), когда прокси и шлюзы могут транслировать отличные от HTTP информационные системы через базовый интерфейс. Каждое сообщение является запросом или откликом. Клиент создаёт запросные сообщения,

указывающие его намерения, и маршрутизирует эти сообщения в направлении идентифицированного сервера-источника. Сервер прослушивает запросы, анализирует каждое принятое сообщение, интерпретирует семантику сообщения применительно к указанному целевому ресурсу и отвечает на запрос одним или несколькими сообщениями с откликами. Клиент проверяет полученные отклики на предмет соответствия его намерениям, определяя дальнейшие действия на основе полученного кода статуса и содержимого.

Стандарт ISO/IEC 19464 регламентирует существование протокола AMQP - открытого протокола прикладного уровня для передачи сообщений между компонентами системы[19]. Основная идея состоит в том, что отдельные подсистемы могут обмениваться произвольным образом сообщениями через AMQP-брокер, который осуществляет маршрутизацию, возможно гарантирует доставку, распределение потоков данных, подписку на нужные типы сообщений. AMQP основан на следующих понятиях:

- Сообщение(message) - единица передаваемых данных, основная его часть (содержание) никак не интерпретируется сервером, к сообщению могут быть присоединены структурированные заголовки.
- Обменник(exchange) - в неё отправляются сообщения. Точка обмена распределяет сообщения в одну или несколько очередей. При этом в точке обмена сообщения не хранятся. Точки обмена бывают трёх типов:
- Очередь(queue) - здесь хранятся сообщения до тех пор, пока не будут забраны клиентом. Клиент всегда забирает сообщения из одной или нескольких очередей.

Сообщение AMQP состоит из набора свойств и непубличного содержимого. Новое сообщение создается Источником(producer) с использованием клиентского API AMQP. Источник добавляет контент в сообщение и, возможно, устанавливает некоторые свойства сообщения. Источник маркирует сообщение с помощью маршрутной информации, которая внешне похожа на адрес, но может быть любой.

Затем Источник отправляет сообщение в Обменник(exchange). Когда сообщение поступает на сервер, Обменник, как правило, направляет его в набор очередей, которые также существуют на сервере. Если сообщение немаршрутизируемо, Обменник может удалить его или вернуть приложению. Источник сам решает, как поступать с немаршрутизируемыми сообщениями. Одно сообщение может существовать во многих очередях сообщений. Сервер может справиться с этим по-разному, например: копирование сообщения с помощью подсчета ссылок и т. д. Это не влияет на интероперабельность. Однако, когда сообщение направляется в несколько очередей сообщений, оно идентично в каждой очереди сообщений. Здесь нет уникального идентификатора, отличающего различные копии. Когда сообщение поступает в очередь сообщений, она немедленно пытается передать его потребителю через AMQP. Если это невозможно, то сообщение хранится в очереди сообщений (в памяти или на диске по просьбе Источник) и ждет, пока Потребитель(consumer) будет готов. Если отсутствует Потребитель, то очередь может вернуть сообщение Источник через AMQP (опять же, если Источник попросил об этом). Когда очередь сообщений может доставить сообщение Потребитель, она удаляет сообщение из своего внутреннего хранилища. Это может произойти сразу же или после того, как Потребитель признает, что он успешно выполнил свою работу, обработал сообщение. Потребитель сам выбирает, как и когда сообщения будут «подтверждены». Потребитель также может отклонить сообщение (отрицательное подтверждение). Сообщения Источника и подтверждения Потребителя сгруппированы в транзакции. Когда приложение играет обе роли, что часто бывает, он выполняет смешанную работу: отправляет сообщения и отправляет подтверждения, а затем фиксация или откат транзакции. Доставка сообщений от сервера к Потребителю не является транзакционной.

2.3. Сравнение и оценка методологических, технологических, алгоритмических и программных решений по теме исследования

2.3.1. Программные решения задачи балансировки и распределения

В разработке программного обеспечения скорость и гибкость — ключевые факторы успеха. Важную роль в реализации этих факторов играют технологии виртуализации и контейнеризации. Причем контейнеризация как более легкая и эффективная альтернатива традиционной виртуализации завоевывает все большую популярность.

Контейнер — это изолированная среда выполнения для приложения. Она включает в себя все необходимое для запуска приложения — исполняемый код, библиотеки, системные инструменты, настройки и зависимости.

Контейнер представляет собой активный процесс, экземпляр образа, запущенный в изолированной среде. Эта изоляция обеспечивается за счет технологий виртуализации на уровне операционной системы, таких как пространства имен и `cgroups` в Linux. `Cgroups` ограничивают ресурсы, доступные контейнеру — CPU, память, дисковый ввод-вывод, предотвращая монополизацию ресурсов одним контейнером и обеспечивая стабильную работу всей системы.

Docker — программное обеспечение для автоматизации развёртывания и управления приложениями в средах с поддержкой контейнеризации, контейнеризатор приложений[20]. Позволяет «упаковать» приложение со всем своим окружением, и зависимостями в контейнер, который может быть развёрнут на любой Linux-системе с поддержкой контрольных групп в ядре, а также предоставляет набор команд для управления этими контейнерами. Изначально использовал возможности LXC, с 2015 года начал использовать собственную библиотеку, абстрагирующую виртуализационные возможности ядра Linux. С появлением Open Container Initiative начался переход от монолитной к модульной архитектуре.

Программное обеспечение функционирует в среде Linux с ядром, поддерживающим контрольные группы и изоляцию пространств имён; существуют сборки только для платформ x86-64 и ARM. Начиная с версии 1.6 (апрель 2015 года) возможно использование в операционных системах семейства Windows.

В состав программных средств входит демон — сервер контейнеров (запускается командой `docker -d`), клиентские средства, позволяющие из интерфейса командной строки управлять образами и контейнерами, а также API, позволяющий в стиле REST управлять контейнерами программно.

Демон обеспечивает полную изоляцию запускаемых на узле контейнеров на уровне файловой системы (у каждого контейнера собственная корневая файловая система), на уровне процессов (процессы имеют доступ только к собственной файловой системе контейнера, а ресурсы разделены средствами `libcontainer`), на уровне сети (каждый контейнер имеет доступ только к привязанному к нему сетевому пространству имён и соответствующим виртуальным сетевым интерфейсам).

Набор клиентских средств позволяет запускать процессы в новых контейнерах (`docker run`), останавливать и запускать контейнеры (`docker stop` и `docker start`), приостанавливать и возобновлять процессы в контейнерах (`docker pause` и `docker unpause`). Серия команд позволяет осуществлять мониторинг запущенных процессов (`docker ps` по аналогии с `ps` в Unix-системах, `docker top` по аналогии с `top` и другие). Новые образы возможно создавать из специального сценарного файла (`docker build`, файл сценария носит название `Dockerfile`), возможно записать все изменения, сделанные в контейнере, в новый образ (`docker commit`). Все команды могут работать как с `docker`-демоном локальной системы, так и с любым сервером Docker, доступным по сети. Кроме того, в интерфейсе командной строки встроены возможности по взаимодействию с публичным репозиторием Docker Hub, в котором размещены предварительно собранные образы приложений, например, команда `docker search` позволяет осуществить поиск среди размещённых в нём

образов[19], образы можно скачивать в локальную систему (docker pull), возможно также отправить локально собранные образы в Docker Hub (docker push).

Rocket (rkt) был разработан CoreOS как альтернатива Docker, ориентированная на безопасность и простоту.[21] В отличие от Docker, rkt не требует демона для управления контейнерами — это упрощает архитектуру и снижает потенциальную поверхность атаки. Rkt использует формат образов appc (Application Container Image) - более открытый и модульный, чем Docker. Хотя rkt не получил такой широкой популярности, он остается жизнеспособным вариантом для тех, у кого в приоритете безопасность и простота.

В рамках рассмотрения микросервисной архитектуры высоконагруженных информационных систем перед разработчиками возникает задача балансировки нагрузки: проблема обеспечения качественной работы программного продукта в условиях ограниченного количества серверных мощностей. К тому же проблема усугубляется неочевидностью распределения этих мощностей по микросервисам. Программным решением данной задачи являются оркестраторы – инструменты для управления контейнерами. Под - это фундаментальная, наименьшая и самая простая единица развертывания оркестратора, которая состоит из одного или нескольких контейнеров. Реплика – это идентичная копия пода.

Основным и самым популярным оркестратором в крупных корпорациях является Kubernetes, разработанный компанией Google[22]. Эта система хорошо известна своей способностью автоматизировать развертывание, управление и, самое главное, масштабирование контейнеров. Если один из них выходит из строя, его место должен занять другой. Kubernetes справляется с этим автоматически, перезапуская, заменяя и уничтожая вышедшие из строя поды, которые не реагируют на проверку работоспособности (см. рис. 2.4).

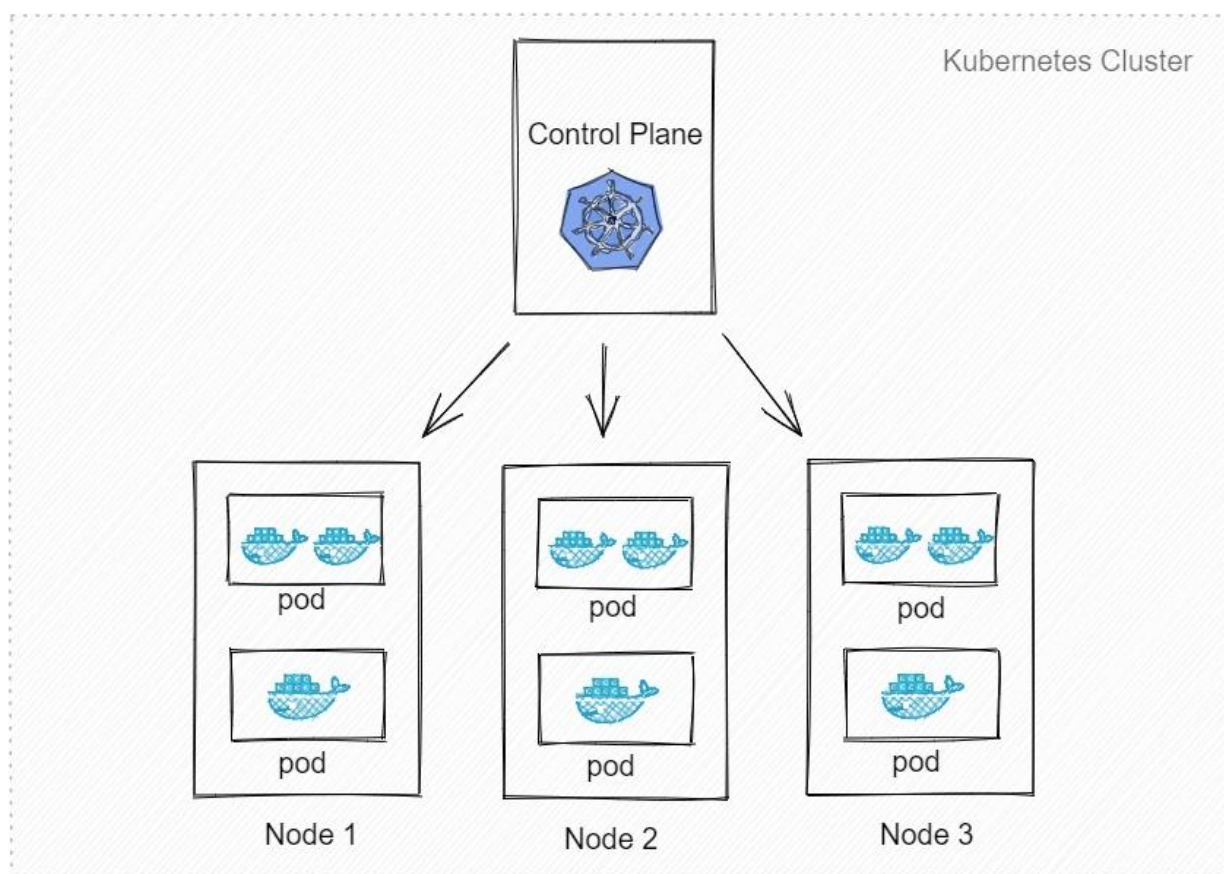


Рисунок 2.4 – Диаграмма работы Kubernetes

К основным преимуществам Kubernetes можно отнести его высокую эффективность и производительность, а также снижение затрат на развертывание и поддержку инфраструктуры. Это проявляется, например, в том, что многие операции, которые раньше выполнялись вручную, Kubernetes автоматизирует. Вместе с тем, одним из основных недостатков этой системы оркестрации можно назвать сложность работы.

Альтернативой Kubernetes является Docker Swarm - встроенный инструмент для оркестрации контейнеров, который разработан специально для экосистемы Docker[23]. Он интегрирован в движок Docker Engine, прост и удобен в использовании. Это делает его привлекательным выбором для команд, уже знакомых с Docker. Swarm хорошо подходит для небольших и средних проектов.

Основное отличие Docker Swarm от Kubernetes заключается в сложности и функциональности. Kubernetes предлагает более мощные возможности для

управления контейнерами, такие как автоматическое масштабирование и управление состоянием приложений, что делает его подходящим для крупных и сложных проектов. В то же время, Docker Swarm проще в использовании и настройке, что делает его идеальным выбором для небольших команд и проектов, где не требуется сложная инфраструктура.

Программным обеспечением, обеспечивающим удобную контейнеризацию является HashiCorp Nomad[24]. Nomad — это современный инструмент для управления контейнерами и оркестрации рабочих нагрузок, разработанный компанией HashiCorp. Он предназначен для упрощения развертывания, управления и масштабирования приложений в различных средах, будь то локальные серверы, облачные платформы или гибридные решения. Nomad поддерживает различные типы рабочих нагрузок, включая контейнеры Docker, виртуальные машины, исполняемые файлы и даже задачи на основе Java. Это делает его универсальным решением для множества сценариев использования.

Nomad отличается высокой производительностью и масштабируемостью, что делает его отличным выбором как для небольших стартапов, так и для крупных корпораций. Он интегрируется с другими инструментами HashiCorp, такими как Consul и Vault, что позволяет создавать комплексные решения для управления инфраструктурой. Благодаря этим интеграциям, Nomad может обеспечить не только оркестрацию рабочих нагрузок, но и управление сервисами и секретами, что делает его мощным инструментом для DevOps-инженеров и системных администраторов.

2.3.2. Алгоритмы балансировки и распределения

В основе программных решений задачи балансировки лежат алгоритмы распределения, которые определяют стратегию отдачи нагрузки на контейнеры/поды.

Алгоритм Round-Robin является статическим алгоритмом распределения нагрузки, который базируется на простой идее, запросы от пользователей распределяются между компонентами системы

(микросервисами) в циклическом порядке, благодаря чему удаётся минимизировать время ожидания ответа клиента[25]. Формализация алгоритма выглядит следующим образом. Пусть имеется N объектов, способных выполнить заданное действие, и M задач, которые должны быть выполнены этими объектами. Подразумевается, что объекты n равны по своим свойствам между собой, задачи m имеют равный приоритет. Тогда первая задача ($m = 1$) назначается для выполнения первому объекту ($n = 1$), вторая — второму и т. д., до достижения последнего объекта ($m = N$). Тогда следующая задача ($m = N+1$) будет назначена снова первому объекту и т. п. Проще говоря, происходит перебор выполняющих задания объектов по циклу, или по кругу (round), и по достижении последнего объекта следующая задача будет также назначена первому объекту. Решение задач может быть дополнительно разбито на кванты времени, причем для продолжения решения во времени нумерация объектов (и, соответственно, назначенные задачи) сдвигается по кругу на 1, то есть задача первого объекта отдается второму, второго — третьему, и т. д., а первый объект получает задачу последнего, либо освобождается для приёма новой задачи. Round Robin прост в реализации и обеспечивает базовую балансировку нагрузки.

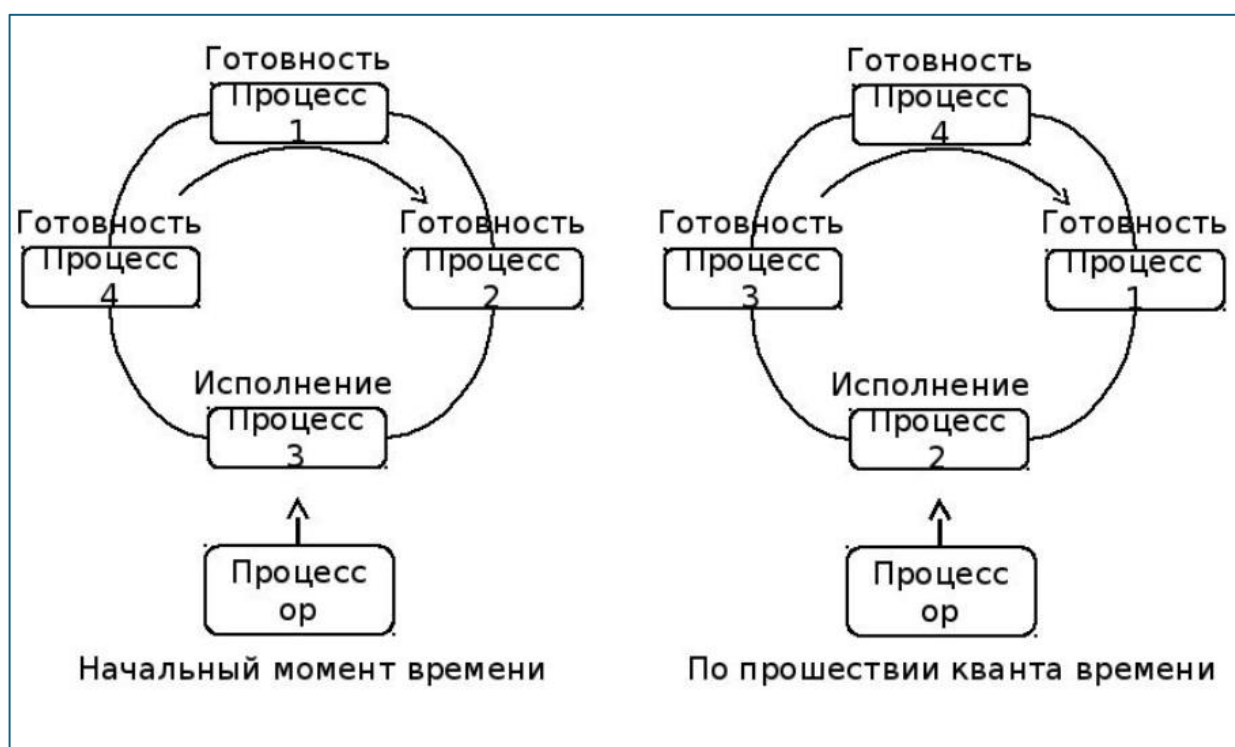


Рисунок 2.5 – Принцип работы алгоритма Round-Robin

Выбор целей осуществляется локально, без взаимодействия с другими балансировщиками нагрузки. Этот же фактор является основным преимуществом данного алгоритма. В общих случаях алгоритм round-robin не обеспечивает высокой производительности.

Least Connections – это алгоритм балансировки нагрузки, который призван выбирать компонент с наименьшим активным соединением из числа доступных. Данный подход ориентирован на учёте текущей нагрузки на компонент и позволяет маршрутизировать запросы.

Работа алгоритма состоит в следующем:

1. При поступлении нового запроса, балансировщик нагрузки анализирует количество активных соединений на каждом из микросервисов в пуле(см. рис 2.6).
2. Запрос направляется к компоненту с наименьшим количеством активных соединений.
3. Счетчик активных соединений обновляется.

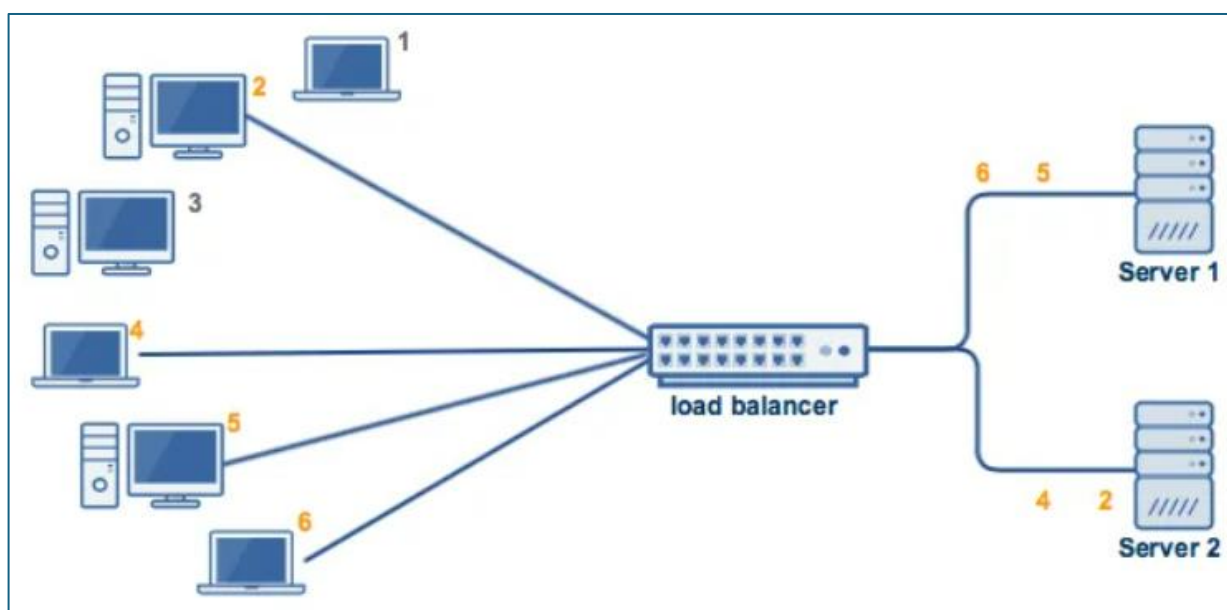


Рисунок 2.6 – Принцип работы алгоритма Least Connections

Least Connections помогает избегать ситуации, при возникновении которых один контейнер оказывается перегруженным длинными задачами, а другие простаивают, потребляя энергию зря.

С другой стороны, этот подход имеет ряд недостатков, которые важно учитывать при его выборе:

- Алгоритм не учитывает реальные характеристики контейнеров, такие как производительность процессора, объем оперативной памяти или пропускная способность. Если один элемент слабее другого, но у него меньше активных соединений, алгоритм все равно направит на него запрос, что приведет к перегрузке.
- Если система обрабатывает множество коротких запросов, алгоритм может не успеть корректно распределить нагрузку. Поды с меньшим количеством активных соединений могут быть перегружены частыми запросами, несмотря на малое число активных соединений в момент принятия решения.

Суть алгоритма IP Hash основана на IP-адресе клиента. Этот метод определяет компонент, на который следует направить запрос, исходя из IP-адреса пользователя. Таким образом, одному и тому же пользователю всегда будет назначен один и тот же компонент, что позволяет сохранить состояние между запросами от одного и того же пользователя.

Работа алгоритма IP Hash следующая:

1. Для каждого входящего запроса определяется IP-адрес клиента.
2. Применяется хеш-функция к IP-адресу, чтобы получить числовое значение (хеш).
3. Хеш значение используется для выбора компонента из пула подов. Обычно используется операция взятия остатка от деления хеша на количество компонентов.

IP Hash может быть особенно полезным в сценариях, где необходимо поддерживать сессию между клиентом и сервером. Например, в веб-

приложениях, где пользователь должен оставаться на одном и том же сервере для сохранения состояния в течение сеанса[26].

Как и у остальных решений, у алгоритма балансировки Nash есть ряд недостатков. В первую очередь это вероятность неравномерного распределения нагрузки в случае, если созданная хэш-функция будет плохо сбалансирована. В результате один узел в составе вычислительного кластера может получать больше нагрузки, чем остальные, что приведет к общему замедлению работы.

Кроме того, Nash не учитывает текущую нагрузку при ее распределении. Если один узел сильно загружен, алгоритм все равно может направить на него запросы, если того вдруг требует хэш. Наконец, если количество подов изменится (например, добавляется новый или один выходит из строя), распределение хэшей может измениться, и многие команды могут начать попадать на другие поды. Это, в свою очередь, нарушит стабильность сессий.

Несмотря на некоторые недостатки, подход Nash полезен в системах, где важна консистентность и делается большая ставка на сохранение данных сессий. Например, в веб-приложениях с высокой нагрузкой, где пользовательская информация сохраняется локально в вычислительном кластере.

Алгоритм наименьшего времени отклика (Least Response Time) является алгоритмом балансировки нагрузки, в основе которого лежит распределение нагрузки постоянный замер времени, который требуется поду для обработки запроса и отправки ответа. При поступлении нового запроса, согласно алгоритму, выбирается под с наименьшим временем измеренного отклика и наименьшим количеством активных соединений, этим обуславливается способность быстро справляться с поступающими командами, учитывая текущую нагрузку. Во многом подобный порядок действий алгоритма позволяет избежать перегрузок и сбоев. Алгоритм Least Response Time лучше всего подходит для систем с неравномерным распределением нагрузки и разными по времени выполнения запросами, где необходимо снизить время

отклика и обеспечить равномерное использование ресурсов. По сравнению со стандартным распределением по количеству соединений, алгоритм наименьшего времени отклика автоматически подстраивается к различным запросам в реальном времени. К минусам алгоритма можно отнести необходимость постоянного мониторинга и оценки времени выполнения запроса каждым подом – от выделенных особенностей зависит эффективность метода, но в то же время сильно усложняется его программно-аппаратная реализация по сравнению с другими методами балансировки.

Случайный выбор между двумя вариантами – подход, в основе которого лежит использование случайности для выбора узлов, однако подбор происходит эффективнее, чем обычный случайный выбор. Сначала балансировщику поступает команда, алгоритм случайным образом выбирает два пода из доступного пула. Затем проводится сравнение текущей нагрузки на двух выбранных узлах по количеству активных соединений или по текущей загрузке. Из двух выбранных подов запрос отправляется на тот, который менее загружен по выбранной характеристике в данный момент. В отличие от обычного случайного распределения, где запрос может попасть на перегруженный под, данный подход значительно повышает вероятность того, что задача будет перенаправлена на более «свободный» для этого узел в данный момент времени. С точки зрения реализации алгоритм относительно прост и не требует сложных вычислений или мониторинга в режиме реального времени всех подов в кластере, как это требуется, например, в рассмотренном ранее алгоритме наименьшего времени отклика.

Основной особенностью алгоритма локальной очереди является динамический процесс. Основная идея - статическое распределение всех новых процессов с миграцией процесса, инициированной хостом, когда его нагрузка падает ниже порогового значения, является определяемым пользователем параметром алгоритма. Параметр определяет минимальное количество готовых процессов, которое диспетчер нагрузки пытается предоставить на каждом процессоре. Он случайным образом отправляет

запросы с количеством локальных готовых процессов удаленным диспетчерам нагрузки. Когда диспетчер загрузки получает такой запрос, он сравнивает локальное количество готовых процессов с полученным числом. Если первое больше, чем второе, то некоторые из запущенных процессов передаются запрашивающей стороне, и возвращается утвердительное подтверждение с количеством переданных процессов.

Алгоритм центральной очереди работает по принципу динамического распределения. Он хранит новые действия и невыполненные запросы в виде циклической очереди FIFO (first-in-first-out) на главном хосте (балансировщике). Каждый новый запрос, поступающий в диспетчер очередей, добавляется в очередь. Затем, когда диспетчер очередей получает запрос на действие, он удаляет первое действие из очереди и отправляет его инициатору запроса. Если в очереди нет задач для выполнения, запрос от вычислительного узла буферизуется до тех пор, пока не станет доступна новая задача. Если новая задача поступает в диспетчер очереди, когда в очереди есть невыполненные задачи, первый такой запрос удаляется из очереди, и ему назначается новая задача. Когда загрузка процессора вычислительного узла падает ниже порогового значения, локальный диспетчер нагрузки отправляет запрос о новом действии в центральный диспетчер нагрузки. Центральный диспетчер нагрузки сразу отвечает на запрос, если в очереди запросов процесса обнаружено готовое действие, или ставит запрос в очередь до тех пор, пока не поступит новое действие.

2.3.3. Основные положения нагрузочного тестирования

Для того чтобы большие информационные системы могли сохранить работоспособность и не деградировать с ростом данных в системе или увеличением числа одновременно работающих пользователей, эти системы должны удовлетворять критериям масштабируемости. Нагрузочное тестирование позволит избежать серьезных проблем в будущем, таких как наличие узких мест, мешающих производительности системы, наличие

архитектурных дефектов в системе, не позволяющих увеличением аппаратных мощностей

увеличить производительность. Данные проблемы, как правило, приводят к тому, что увеличение аппаратных мощностей не дает пропорционального увеличения производительности, а исправление таких проблем возможно только с изменением архитектуры, что ведет к переписыванию части компонент или самой информационной системы.

Нагрузочное тестирование (Load Testing) оценивает производительность системы под определенной нагрузкой. Цель теста — проверить, как приложение или сервер реагируют на большое количество запросов, обрабатывают данные и поддерживают стабильную работу. Результаты тестирования помогают разработчикам и администраторам выявить слабые места продукта и оптимизировать его до запуска.

Функциональное тестирование - это проверка того, корректно ли выполняет программа или приложение свои функции. Основными аспектами функционального тестирования являются либо основания на требованиях или основание на бизнес-процессах. Нефункциональное тестирование - это тестирование, которое сосредотачивается на исследовании тех свойств приложения, которые не относятся к основным функциям: надёжность, комфортность обслуживания, эффективность, совместимость, производительность, безопасность, удобство и мобильность.

Ramp up - это период тестирования, в течение которого нагрузка на систему увеличивается от нуля или минимального значения до целевого уровня, который планируется тестировать. Длительность ramp-up зависит от целей тестирования и особенностей системы. Необходимо такое время для ramp-up, чтобы сервис успел пройти все этапы прогрева, а именно установку соединений, наполнение кэша, запуск фоновых процессов. Только после этого можно считать, что система работает в стабильном режиме, и снимать интересующие показатели. В реальной жизни пользователи подключаются к сервису постепенно, поэтому имитация нагрузки должна быть постепенной,

чтобы сервис успевал адаптироваться к росту запросов, и чтобы метрики отражали именно ту работу, которую система выполняет в обычной эксплуатации, а не в момент резкого старта.

Если сразу подать большую нагрузку, система окажется в ситуации, с которой она сталкивается значительно короткое время своей эксплуатации, что может привести к неестественным сбоям или, наоборот, к излишнему резервированию ресурсов, которые в реальной жизни никогда не понадобятся. Показатели программно-аппаратного обеспечения, снятые в этот момент, не покажут реальную производительность системы, потому что любые задержки или ошибки будут связаны не столько с эффективностью системы, а столько с её подготовкой к работе. По этой же причине не стоит обращать внимание на показатели в первые минуты теста, так как чаще всего они не отражают стабильную картину и реальную производительность. Чтобы получить объективную оценку работы сервиса, важно анализировать именно ту часть теста, когда нагрузка постоянная и сервис работает в условиях, максимально приближённых к реальной эксплуатации.

Ramp down - это заключительная фаза нагрузочного теста, в течение которой нагрузка постепенно снижается с максимального уровня до нуля или минимального порогового значения: в реальных условиях пользователи редко покидают сервис все одновременно - их активность уменьшается постепенно.

Продолжительность фазы ramp down, как правило, составляет от одной до нескольких минут. Показатели, которые снимаются во время ramp-down, не отображают реальную производительность сервиса под нагрузкой, потому что в этот момент системе становится легче: часть пользователей уже отключилась, ресурсы постепенно освобождаются, могут разгружаться очереди, очищаться кэш, поэтому для объективной оценки состояния системы на пике нагрузки всегда стоит выносить ramp-down за скобки: анализировать только стабильную фазу, когда нагрузка не меняется.

Стабильная фаза нагрузки (steady load phase) – это основной этап нагрузочного тестирования, в ходе которого система работает под постоянным,

заранее заданным уровнем нагрузки. Именно этот период нагрузочного тестирования отражает реальную отказоустойчивость и способность справляться с ожидаемым потоком запросов в течение продолжительного времени. Продолжительность данной фазы составляет минимум 5 минут, но длительность может быть больше в зависимости от целей тестирования. На рисунке 2.7 представлен график нагрузки во времени.

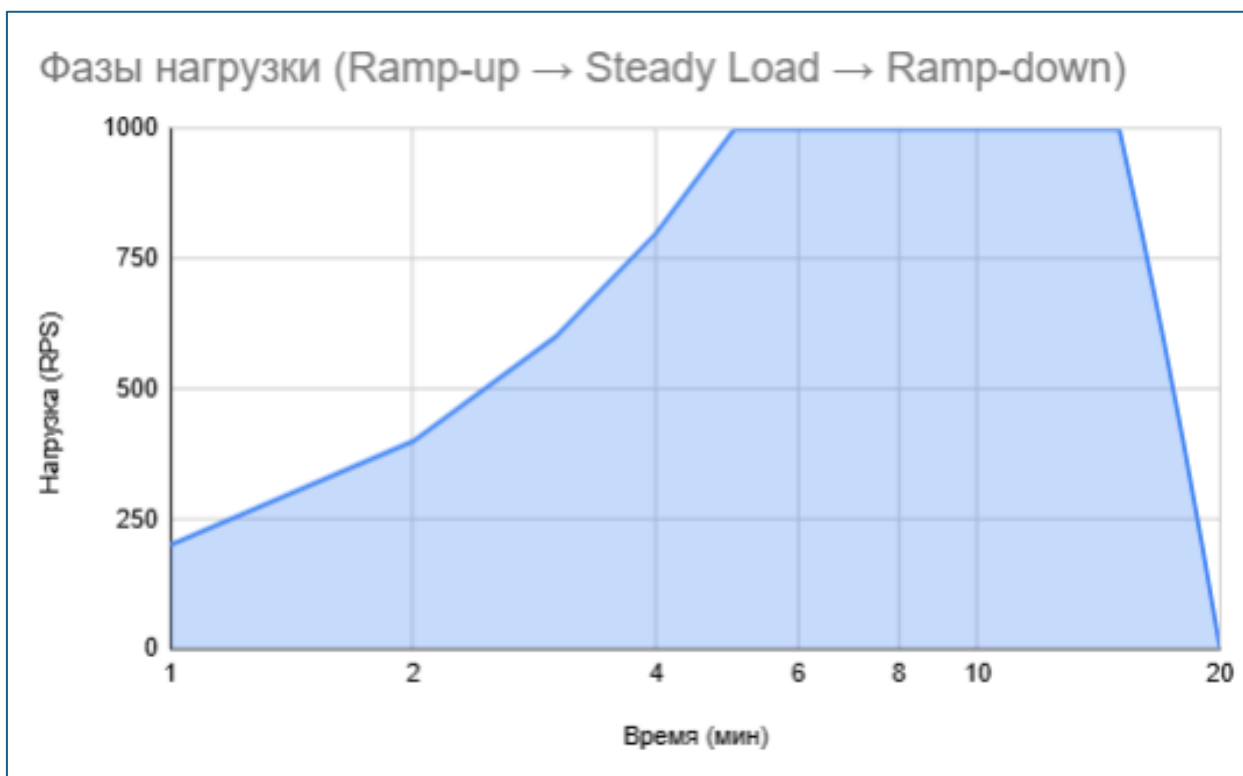


Рисунок 2.7 – График нагрузки во времени

Сервис всегда должен пройти этап прогрева: установление соединений, заполнение кэша, разогрев очередей, первичная инициализация базы данных. Если сразу начать собирать метрики после старта нагрузки, то в них попадут артефакты, связанные именно с этим прогревом, а не с реальной производительностью. (см. рис. 2.8).

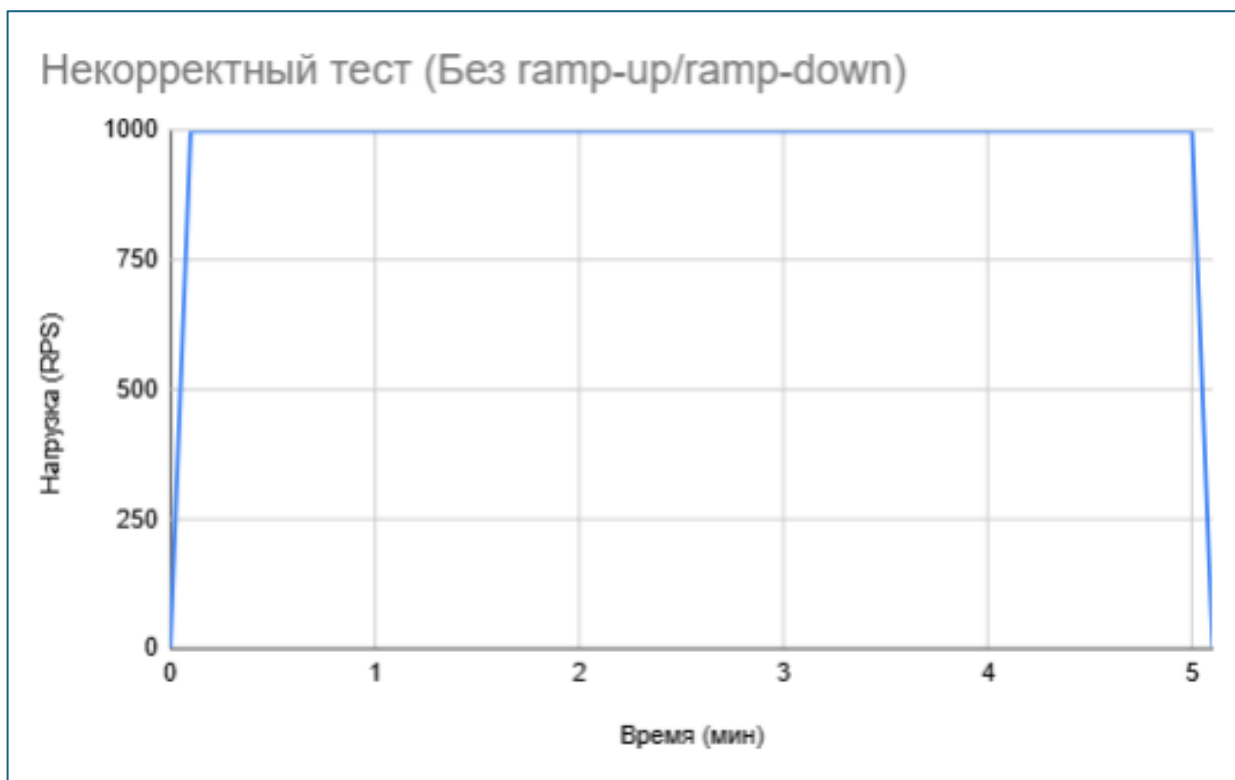


Рисунок 2.8 – График нагрузки во времени неправильного сценария тестирования

Некоторые ограничиваются тестами, которые длятся всего 2-3 минуты, и делают выводы на основе этих данных. В такие короткие промежутки система часто не успевает полностью прогреться, не выявляются проблемы с долгосрочной деградацией, а показатели задержек бывают заниженными просто потому, что сервис ещё не столкнулся с высокой реальной нагрузкой[28].

Одна из самых частых ошибок — рассчитывать средние показатели (latency, RPS, ошибки) по всему тесту, не разделяя фазы ramp-up, steady load и ramp-down. В результате такие усреднённые значения кажутся привлекательными, но на деле они не показывают настоящую устойчивость системы под постоянной нагрузкой. Это может привести к ложному ощущению стабильной работы сервиса, хотя на самом деле в стабильной фазе уже возникают задержки или ошибки(см. рис. 2.9).

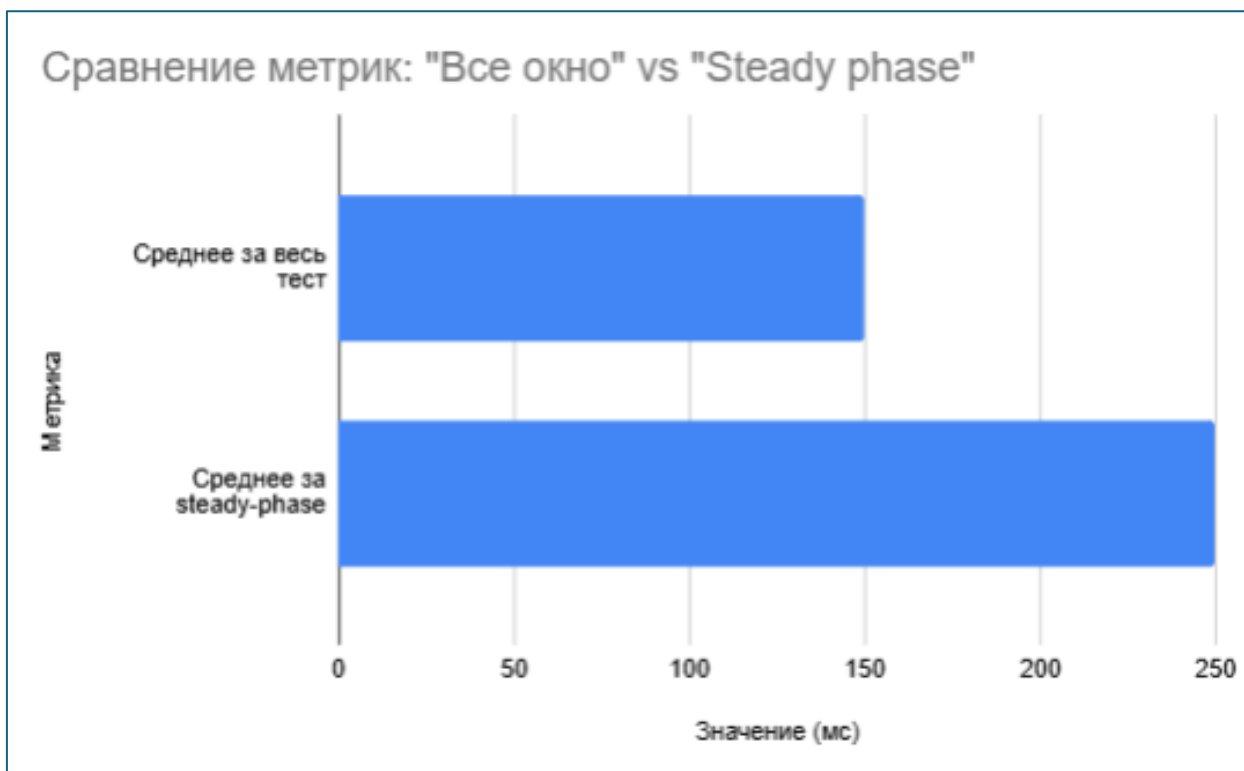


Рисунок 2.9 – Некорректное сравнение метрик.

Иногда для теста выбирают сценарий стресса — сразу выставляют максимальное количество запросов без ramp-up, чтобы проверить устойчивость системы в экстремальных условиях. Но потом анализируют метрики так, будто это был обычный нагрузочный тест, ожидая увидеть стабильную работу. В результате получаются некорректные выводы: стресс-тест предназначен для поиска пределов прочности, а не для оценки повседневной производительности.

Для корректного планирования нагрузки важно продумывать сценарии, отделять разные фазы теста, анализировать только стабильную часть, давать системе время на прогрев и длительную работу, а также внимательно следить за всеми отклонениями и ошибками, которые могут появиться в процессе тестирования.

2.3.4. Инструменты нагрузочного тестирования

Apache JMeter - инструмент для проведения нагрузочного тестирования, разрабатываемый Apache Software Foundation[29]. Хотя изначально JMeter разрабатывался как средство тестирования web-приложений, в настоящее

время он способен проводить нагрузочные тесты для JDBC-соединений, FTP, LDAP, SOAP, JMS, POP3, IMAP, HTTP и TCP. Интересна возможность создания большого количества запросов с помощью нескольких компьютеров при управлении этим процессом с одного из них. Архитектура, поддерживающая плагины сторонних разработчиков, позволяет дополнять инструмент новыми функциями.

В программе реализованы механизмы авторизации виртуальных пользователей, поддерживаются пользовательские сеансы. Организовано логирование результатов теста и разнообразная визуализация результатов в виде диаграмм, таблиц и т. п.

HPLoadRunner (также HPELoadRunner) — утилита для автоматизированного нагрузочного тестирования[30]. Модуль Virtual User Generator — служит для разработки скриптов, которые будут задействованы для дальнейшего тестирования. Имеет большой набор инструментов, помогающих написать максимально продуктивные скрипты для тестирования приложения. Часть инструментов позволяет вести автоматическое написание скриптов. Достаточно включить «запись» и все действия, выполняемые пользователем на компьютере, будут записываться в скрипт (своего рода «логирование»). Хотя в дальнейшем такие скрипты желательно вручную доработать, исправить или оптимизировать, повышая тем самым их эффективность и безотказность. Модуль Controller — основной модуль программы. Выполняет сценарии проведения тестирования по заданным настройкам. В этот модуль включаются скрипты, написанные в Virtual User Generator. Администратор имеет возможность создать сценарий тестирования. Модуль Analysis — служит для составления детальных отчётов о проделанном тестировании

Также данный модуль имеет функции для настройки работы с параметрами защиты тестируемого приложения. Допустим, если трафик сайта защищён недоверенным сертификатом, то при входе на такой сайт защита будет выдавать предупреждение о том, что надёжность сайта подозрительна.

В результате настроек HP LoadRunner для работы с таким сертификатом в автоматическое написание скриптов не будут попадать лишние данные о защите сайта, что существенно улучшит работу скрипта. Скрипты, созданные данным модулем, имеют гибкую структуру, которую можно настраивать в зависимости от требований к тесту.

2.4. Постановка задачи исследования

Исходя из того, что у каждого проанализированного алгоритма балансировки есть как преимущества, так и недостатки, и особенности работы в той или иной ситуации, важно перед выбором конкретного метода определить, какой из вариантов лучше подойдет для вашего случая. В том числе важно обратить внимание на общее количество запросов, скорость их обработки, наличие и частота наплывов пользователей (например, связанные с сезонностью или временем суток).

К тому же прямой комплексной оценки архитектуры не существует. Сложно однозначно сказать, выдержит ли наша система нагрузку в час-пик, когда она испытывает наибольшую нагрузку, а также провести масштабирование/демасштабирование системы на основе увеличения/уменьшения популярности страниц(эндпоинтов) у пользователей.

Существующие в настоящее время подходы к балансировки информационной системы на основе микросервисного паттерна далеки от совершенства, они не оптимизируют работу информационной системы по соотношению «ресурсы-качество», не обладают методологическим единством, а также характеризуются появлением новых идей и технологий, нуждающихся в обобщении и развитии, и не учитывают специфику высоконагруженной с точки зрения запросов пользователей туристической отрасли. В данных обстоятельствах становятся необходимым постановка и решение актуальной научной задачи - выявление неявных закономерностей и определение оптимальной архитектуры микросервисного приложения, которые повысят эффективность информационных систем.

Цель магистерской диссертации – исследование методов, которые помогут повысить эффективность информационных систем в сфере туризма по таким критериям, как производительность, так и отказоустойчивость.

Следующие **задачи** определены для достижения поставленных целей:

- Проведение анализа предметной области и выявление специфических требований к информационным системам в сфере туризма.
- Исследование архитектурных подходов к построению информационной системы, систематизировать их достоинства, недостатки и области применения.
- Разработка методики выбора структурного типа архитектуры информационной системы на основе анализа ключевых параметров проекта.
- Проектирование архитектурную модель информационной системы на основе микросервисного подхода.

3 ОБЗОР ИЗВЕСТНЫХ МЕТОДОВ И СРЕДСТВ РЕШЕНИЯ ПРОБЛЕМЫ

3.1. Основные положения методики

Изначальные технические(физические) условия: Существует сервер, имеющий 16 ГБ ОЗУ и процессор с 8 ядрами.

Основной стек приложения:

- Backend – php 8.2 & laravel
- Frontend – vue 3
- БД – mysql (mariadb)
- Redis
- RabbitMQ
- Elasticsearch & Prometheus
- Docker & Kubernetes

Планируется создание *микросервисного приложения*, состоящего из $N = \text{const}$ сервисов, у каждого сервиса (пода) будет i реплик (копий) сервиса, где $i = [1, K]$, где K – заранее оговоренное некоторое число, зависящее от тестов. Каждой реплике (копии) отводится постоянное пространство памяти. Ключевой особенностью сбора метрик заключается в анализе относительных, а не абсолютных значений. Введём новые термины:

- *Матрица отношений R* отражает характеристику обращений микросервисов друг к другу посредством API.
- *Под* - это фундаментальная, наименьшая и самая простая единица развертывания. В рамках нашей методики система построена по принципу: «один сервис – один под».
- *Реплика* – это идентичная копия пода.
- *si (swap in)* - количество памяти, загружаемой обратно из swap в оперативную память (КБ/сек)
- *so (swap out)* - количество памяти, выгружаемой из оперативной памяти в swap (КБ/сек)

- *Load Average* — это среднее количество процессов, которые находятся в состоянии готовности к выполнению или ожидания I/O (uninterruptible sleep) за последние 1, 5 и 15 минут.
- *Статичными данными* будем называть данные, собранные экспериментальным путём с пары «система-сервер».
- *Практическими данными* будем называть данные, полученные экспериментальным путём с пары «система-сервер» под действием нагрузочных тестов.

Основной *идеей* методики является ***итеративное увеличение количества реплик сервисов при проведении нагрузочных тестов и последующий анализ собранных метрик.***

Основными *целями* данной методики являются ***выявление неявных закономерностей и определение оптимальной архитектуры микросервисного приложения.***

3.2. Основные принципы нагрузочного тестирования

В рамках методики будет проводится нагрузочное тестирование N микросервисов.

Нагрузочным тестом микросервиса X называется вектор $L = (L_{\text{get}}, L_{\text{post}}, L_{\text{put}}, L_{\text{patch}}, L_{\text{delete}})$ — где каждый элемент вектора представляет собой количество обращений в секунду (RPS) к эндпоинту соответствующего HTTP-метода.

Нагрузочным тестированием при заданном количестве реплик K будем называть упорядоченную конечную последовательность тестов $L_1 \leq \dots \leq L_n$ продолжительностью t секунд, где n и t — заранее оговоренные числа. ($\leq$ обозначает что элементы вектора не меньше предыдущего). В рамках эксперимента проводится K нагрузочных тестирований, для каждого количества реплик.

Критической нагрузкой $L_{\text{крит}}$ будем называть такую нагрузку, что приводит к качественному ухудшению работы системы. Это точка с момент, когда система перестает справляться с нагрузкой *комфортно* и начинает

деградировать по ключевым метрикам. Её также называют точкой перегиба или точкой насыщения.

В предварительном этапе проводится измерение статичных данных:

- Статичная матрица задержки сети $M_{\text{стат.сеть}}$ - симметричная матрица размерности $N*N$, элементы отображают задержку сети между сервисами, по диагонали ставятся 0, так как задержка внутри отдельного сервиса равна 0.
- Относительная статичная загрузка памяти $MEM_{\text{стат}}$ (вектор значений размерности N , с элементами от 0 до 1);
- Относительная статичная загрузка процессора $CPU_{\text{стат}}$ (вектор значений размерности N , с элементами от 0 до 1);
- Абсолютное статичное энергопотребление системы $E_{\text{стат}}$ (Квт*ч).

Для системы в целом, и для отдельных сервисов, проводятся нагрузочные тесты, в результате которых логируются следующие величины:

- Нагрузочные тесты L
- Относительная загрузка памяти MEM_i (вектор значений размерности N , с элементами от 0 до 1);
- Относительная загрузка процессора CPU_i (вектор значений размерности N , с элементами от 0 до 1);
- Абсолютное энергопотребление системы E_i (Квт*ч) //спорная величина;
- Матрица задержки сети $M_{\text{сеть.i}}$ (симметричная матрица размерности $N*N$, элементы отображают задержку сети между сервисами).
- Матрица времени отклика между отправкой запроса и получением ответа — $M_{\text{load.i}}$ (матрица размерности $N*N$, элементы которой отображают время между отправкой запроса и получением ответа).

Основными исследуемыми характеристиками являются:

1. Использование памяти.
2. Потребление ресурса процессора.
3. Энергопотребление.

4. Стабильность сети.
5. Производительность.
6. Характеристика обмена в RabbitMQ.
7. Error Rate
8. Общие взаимосвязи.

3.3. Использование памяти

В рамках данной характеристики рассмотрим анализ одного микросервиса (принцип для остальных – тот же самый). Нагрузка на память сервера зависит от нагрузки на сервис, но эта зависимость часто имеет ступенчатый, пилообразный характер и сильно зависит от характера работы сервиса с памятью.

В рамках тестирования на вход сервису подаётся нагрузка вектор $L_i = (L_{\text{get}}, L_{\text{post}}, L_{\text{put}}, L_{\text{patch}}, L_{\text{delete}})$, каждые t/r мс считывается вектор относительной загрузки памяти $\text{MEM}_{i,r}$. Вычислим для каждого L_i среднее квадратичное отклонение набора значений $\text{MEM}_{i,r}$. Так как $\text{ср.кв.откл} \ll \text{MEM}_{i,r}$, то мы можем говорить что для данного сервиса $\text{MEM}_i = \text{const}$. Рассчитаем значение MEM_i как среднее арифметическое значение набора $\text{MEM}_{i,r}$.

Проведя для данного микросервиса с данным количеством реплик n тестов: $L_1 \leq \dots \leq L_n$ получим закономерность вида $f(L_{\text{get}}, L_{\text{post}}, L_{\text{put}}, L_{\text{patch}}, L_{\text{delete}}) = \text{MEM}$. Для установления явного биективного отношения проведем нормализацию:

Пусть w_0 – базовое потребление(нагрузка) памяти, w_1, w_2, w_3, w_4, w_5 – некоторые веса, тогда

$$w_0 + w_1 * L_{\text{get}} + w_2 * L_{\text{post}} + w_3 * L_{\text{put}} + w_4 * L_{\text{patch}} + w_5 * L_{\text{delete}} = \text{MEM}$$

вычислив веса w_1, w_2, w_3, w_4, w_5 – путём явной подстановки нагрузочных параметров.

Таким образом, получив значения весов можем перейти к функции одной переменной:

$$f(L) = \text{MEM}$$

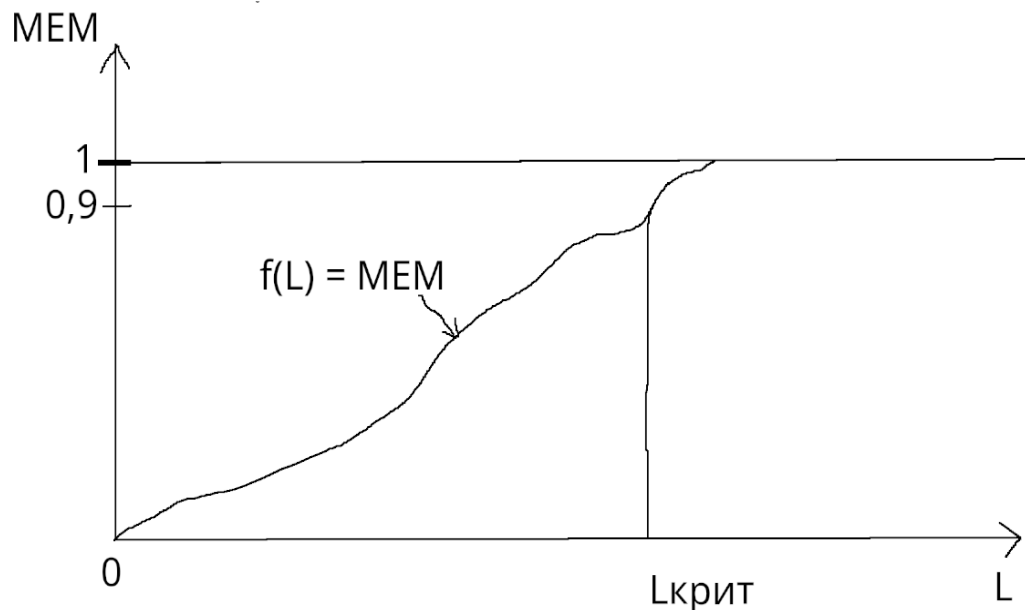


Рисунок 3.1 - Функция $f(L) = \text{MEM}$

Аналогичным способом мы можем собрать данные по использованию своппинга (si , so):

$$\text{SWAP} = (\text{swap_used} / \text{swap_total})$$

Используя вышеописанный метод нормализации параметров с помощью весов, мы получим явную взаимосвязь вида:

$$g(L) = \text{SWAP}$$

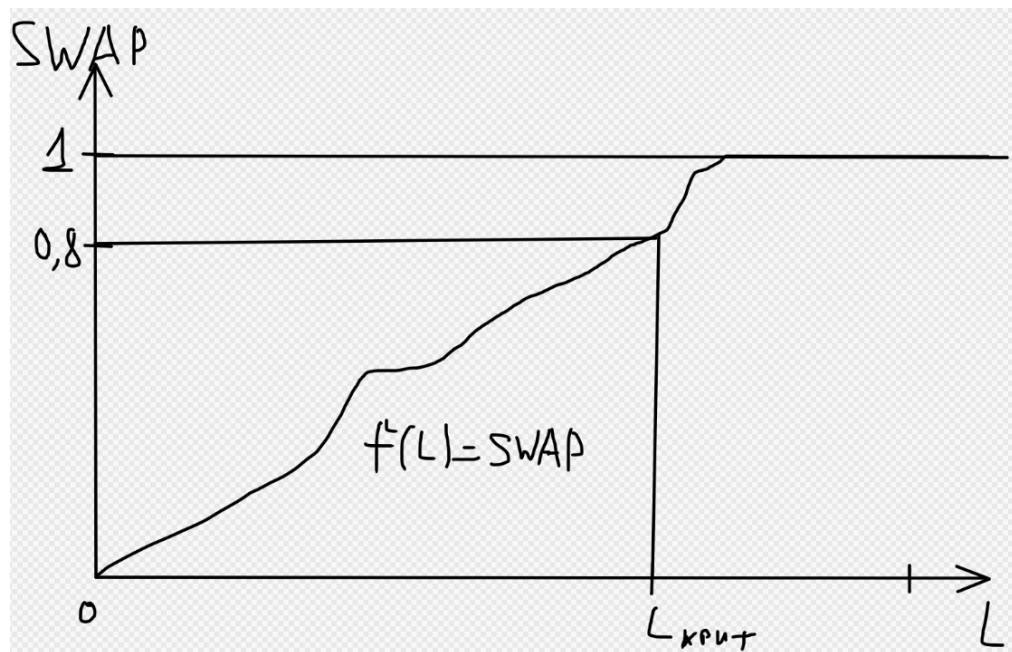


Рисунок 3.2 - Функция $g(L) = \text{SWAP}$

Получив для заданного количества реплик сервиса данные об использовании памяти, мы можем выявить критическую нагрузку пода. Критическая нагрузка на память наступает, когда наблюдается:

- Устойчивое использование RAM (в данном случае $MEM > 0.9$).
- Активное использование своппинга:

$$SWAP > 0.8 \text{ и } si + so > 1000 \text{ KB/s}$$

Критическая нагрузка рассчитывается по правилу наименьшего:

$$L_{\text{крит}} = \min(L_{\text{крит.своп}}, L_{\text{крит.озу}})$$

Составим матрицу размерности $N \times k$, где N - количество микросервисов, k - количество реплик микросервисов, элементы матрицы отображают *критическую нагрузку* обрабатываемое микросервисом N с количеством реплик k . Данная матрица отражает характеристику использования памяти микросервисного приложения.

Назовём этой матрицу RES_{mem} . Выведем из этой матрицы нагрузочные характеристики:

По каждой строке вычислим разностную характеристику:

$$DIFF_{\text{mem}}(i, j) = (RES_{\text{mem}}(i, j + 1) - RES_{\text{mem}}(i, j)) / RES_{\text{mem}}(i, 1)$$

Разностная характеристика определяется по наименьшему значению. В дальнейшем этот факт будет использоваться в анализе общих характеристик.

3.4. Использование ресурсов процессора

В рамках данной характеристики рассмотрим анализ одного микросервиса (принцип для остальных – тот же самый). Нагрузка на процессор сервера зависит от нагрузки на сервис, но эта зависимость как правило носит хаотичный, нелинейный характер.

В рамках тестирования на вход сервису подаётся нагрузка вектор $L_i = (L_{\text{get}}, L_{\text{post}}, L_{\text{put}}, L_{\text{patch}}, L_{\text{delete}})$, каждые t/r мс считывается вектор относительной загрузки процессора $CPU_{i,r}$. Вычислим для каждого L_i среднее квадратичное отклонение набора значений $CPU_{i,r}$. Так как $\text{ср.кв.откл} \ll CPU_{i,r}$, то мы можем говорить что для данного сервиса $CPU_i = \text{const}$. Рассчитаем значение CPU_i как среднее арифметическое значение набора $CPU_{i,r}$.

Проведя для данного микросервиса с данным количеством реплик n тестов: $L_1 \leq \dots \leq L_n$ получим закономерность вида $f(L_{\text{get}}, L_{\text{post}}, L_{\text{put}}, L_{\text{patch}}, L_{\text{delete}}) = \text{CPU}$. Для установления явного биективного отношения проведем нормализацию:

Пусть w_0 – базовое потребление(нагрузка) процессора, w_1, w_2, w_3, w_4, w_5 – некоторые веса, тогда

$$w_0 + w_1 * L_{\text{get}} + w_2 * L_{\text{post}} + w_3 * L_{\text{put}} + w_4 * L_{\text{patch}} + w_5 * L_{\text{delete}} = \text{CPU}$$

вычислив веса w_1, w_2, w_3, w_4, w_5 – путём явной подстановки нагрузочных параметров.

Таким образом, получив значения весов можем перейти к функции одной переменной:

$$f(L) = \text{CPU}$$

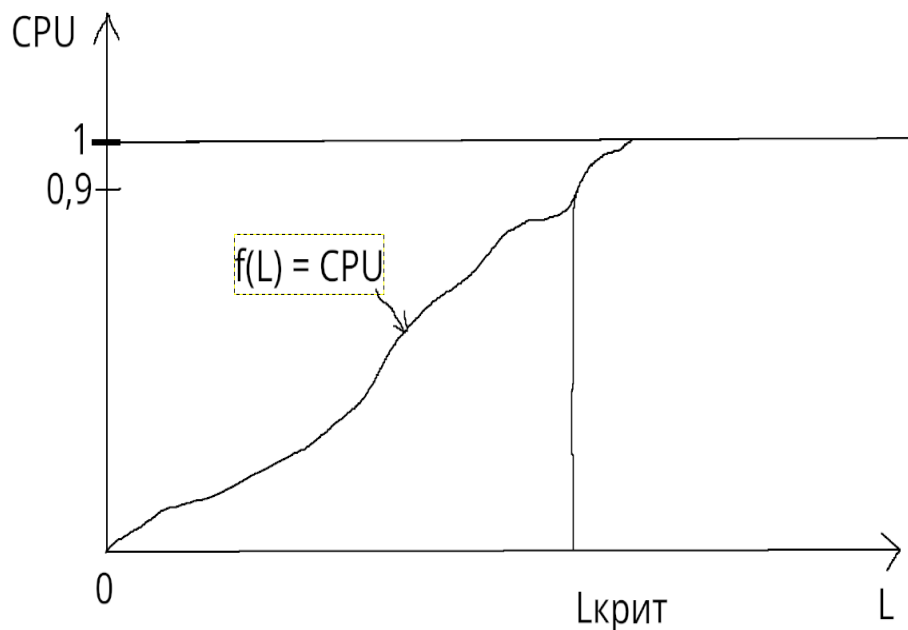


Рисунок 3.3 - Функция $f(L) = \text{CPU}$

Так же собираются метрики по средней загрузке (Load Average) - показателю, который отражает, насколько загружена система процессами, которые либо используют процессор, либо ожидают его освобождения.

$$g(L) = \text{Load Average}$$

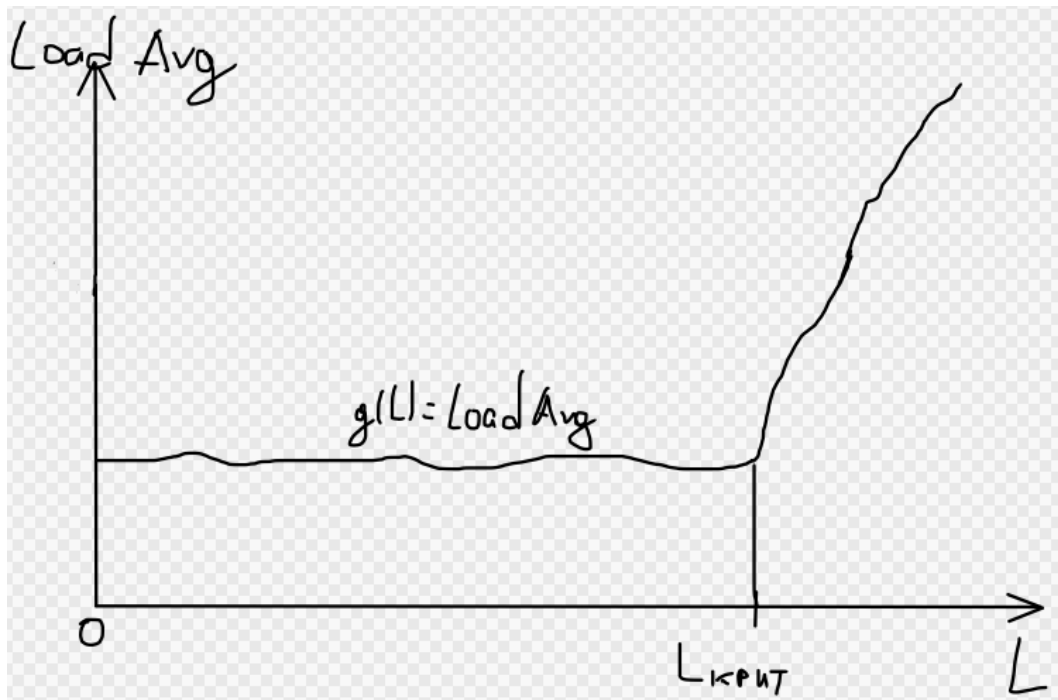


Рисунок 3.4 - Функция $g(L) = \text{Load Average}$

Получив для заданного количества реплик сервиса данные об использовании процессора, мы можем выявить критическую нагрузку пода. Критическая нагрузка на процессор наступает, когда наблюдается:

- Нагрузка на процессор достигает $\text{CPU} > 0.8-0.9$;
- Растет **Load Average**;

Критическая нагрузка рассчитываем по правилу наименьшего:

$$L_{\text{крит}} = \min(L_{\text{крит.cpu}}, L_{\text{крит. load_avg}})$$

Составим матрицу размерности $N \times k$, где N - количество микросервисов, k - количество реплик микросервисов, элементы матрицы отображают *критическую нагрузку* обрабатываемое микросервисом N с количеством реплик k . Данная матрица отражает характеристику использования процессора микросервисного приложения.

Назовём этой матрицу RES_{cpu} . Выведем из этой матрицы нагрузочные характеристики:

По каждой строке вычислим разностную характеристику:

$$\text{DIFF}_{\text{cpu}}(i, j) = (\text{RES}_{\text{cpu}}(i, j + 1) - \text{RES}_{\text{cpu}}(i, j)) / \text{RES}_{\text{cpu}}(i, 1)$$

Разностная характеристика определяется по наименьшему значению. В дальнейшем этот факт будет использоваться в анализе общих характеристик.

3.5. Энергопотребление

В рамках данной характеристики рассмотрим анализ одного микросервиса (принцип для остальных – тот же самый). Энергопотребление сервера зависит от нагрузки на компоненты и на сервис в целом, но эта зависимость как правило носит хаотичный характер.

В рамках тестирования на вход сервису подаётся нагрузка вектор $L_i = (L_{get}, L_{post}, L_{put}, L_{patch}, L_{delete})$, каждые t/r мс считывается вектор мощности P_i . Вычислим для каждого L_i энергопотребление сервиса по времени t :

$$E = \int_0^t (P_i(t) dt)$$

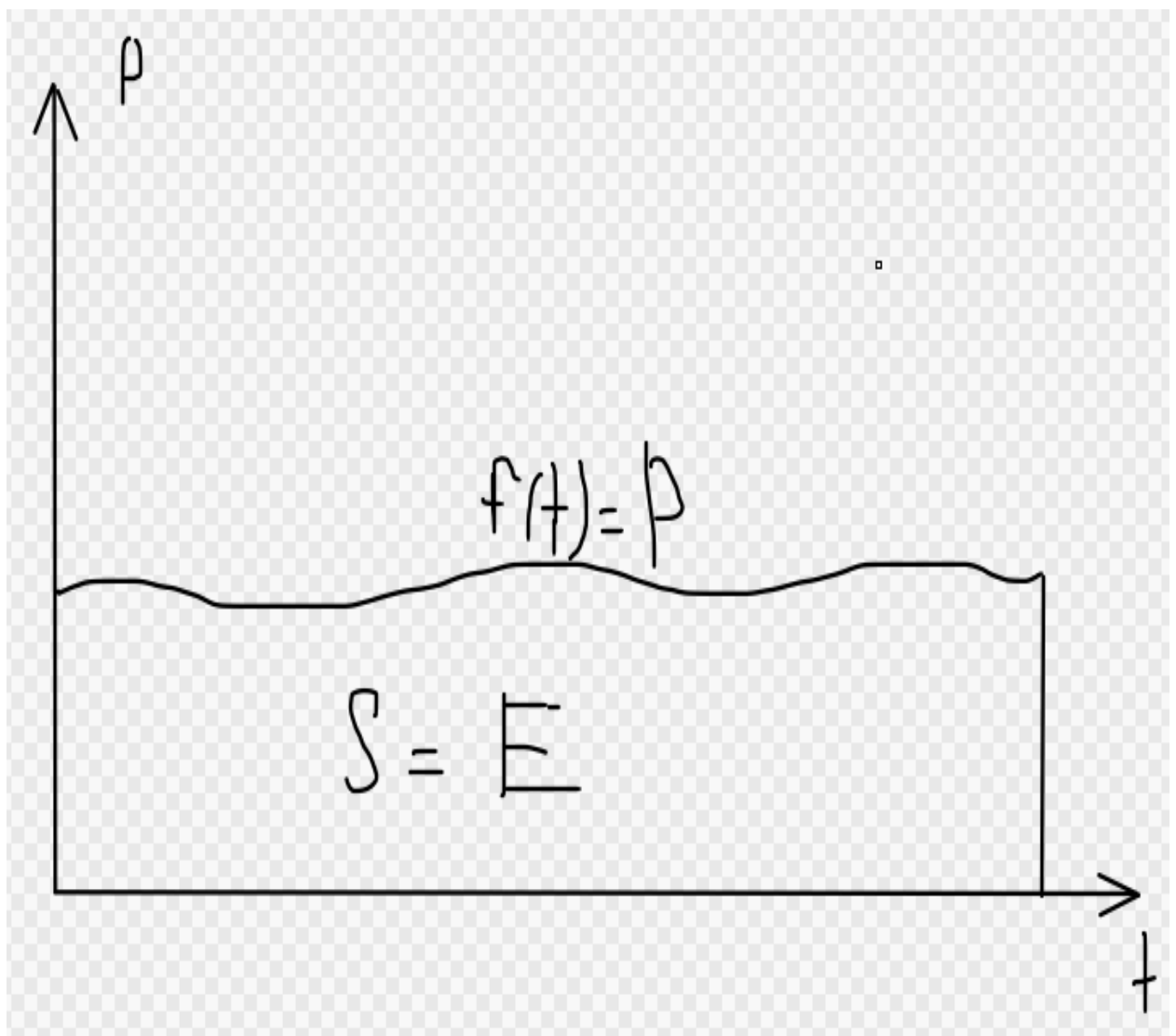


Рисунок 3.5 - Функция энергопотребления

Проведя для данного микросервиса с данным количеством реплик n тестов: $L_1 \leq \dots \leq L_n$ получим закономерность вида $f(L_{get}, L_{post}, L_{put}, L_{patch}, L_{delete})$

= E. Для установления явного биективного отношения проведем нормализацию:

Пусть w_0 – базовое потребление, w_1, w_2, w_3, w_4, w_5 – некоторые веса, тогда

$$w_0 + w_1 * L_{\text{get}} + w_2 * L_{\text{post}} + w_3 * L_{\text{put}} + w_4 * L_{\text{patch}} + w_5 * L_{\text{delete}} = E$$

вычислив веса w_1, w_2, w_3, w_4, w_5 – путём явной подстановки нагрузочных параметров.

Таким образом, получив значения весов можем перейти к функции одной переменной:

$$f(L) = E$$

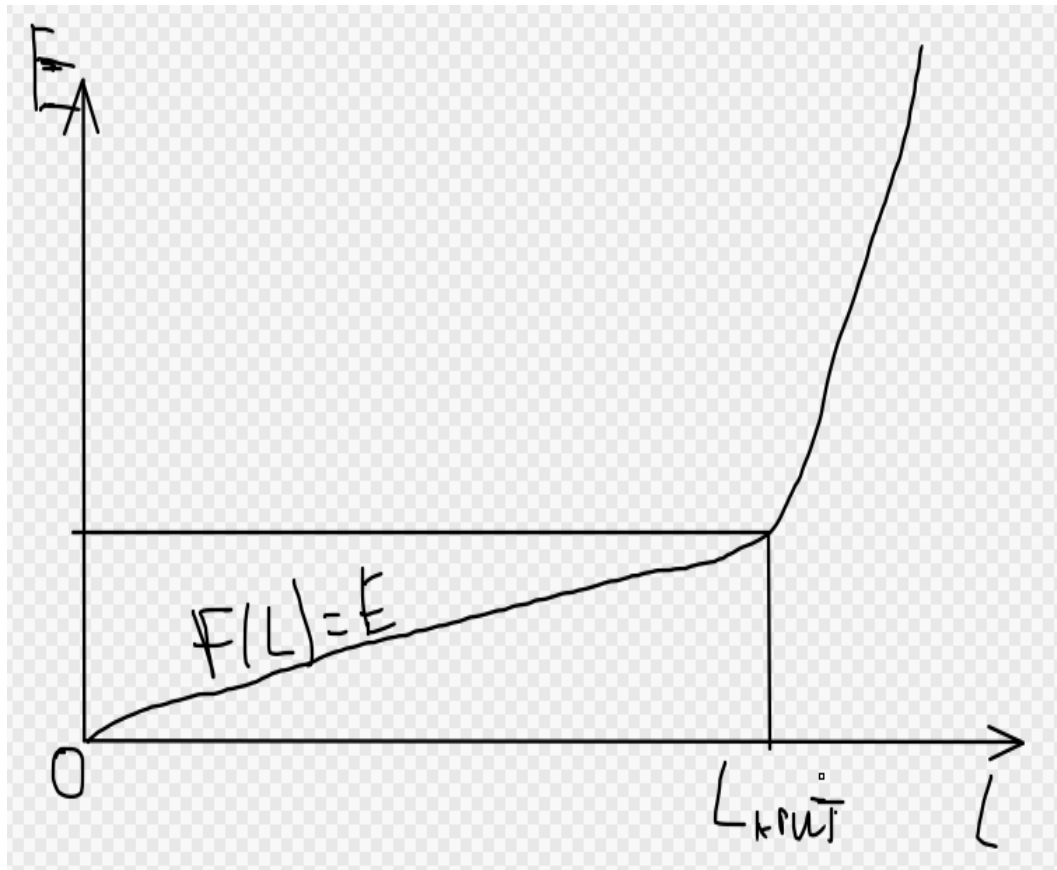


Рисунок 3.6 - Функция $f(L) = E$

А также вычислим:

$$f'(L) = dL/df$$

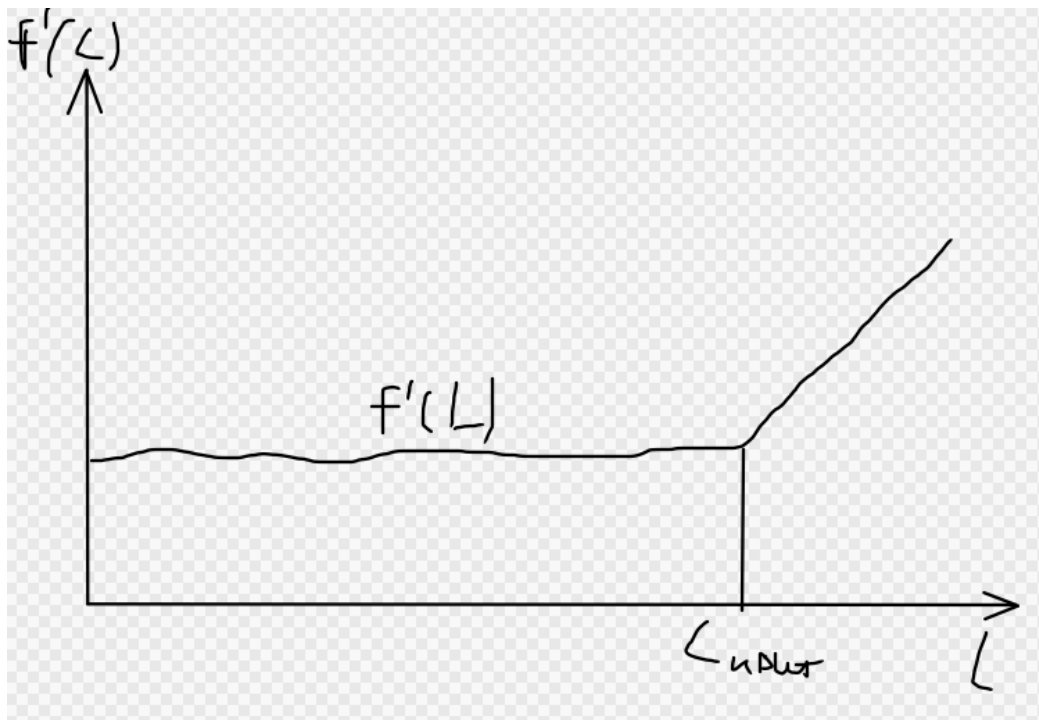


Рисунок 3.7 - Функция $f'(L) = dL/df$

Оценка:

- функция $f(L)$ показывает зависимость энергопотребления от заданной нагрузки. *Ожидается линейная зависимость, которая потом перейдёт в степенную или экспоненциальную.*
- функция $f'(L)$ показывает рост энергопотребления от нагрузки. *Ожидается константная зависимость, которая потом перейдёт в линейную, степенную или экспоненциальную.*

Получив для заданного количества реплик сервиса данные об энергопотреблении, мы можем выявить критическую нагрузку пода. Критическая нагрузка в контексте энергопотребления наступает, когда наблюдается резкий рост функции $f'(L)$ (энергопотребление начинает увеличиваться с «большим» темпом, нежели чем как раньше).

Составим матрицу размерности $N \times k$, где N - количество микросервисов, k - количество реплик микросервисов, элементы матрицы отображают *критическую нагрузку* обрабатываемое микросервисом N с количеством реплик k . Данная матрица отражает характеристику энергопотребления микросервисного приложения.

Назовём этой матрицу RES_E . Выведем из этой матрицы нагрузочные характеристики:

По каждой строке вычислим разностную характеристику:

$$DIFF_E(i, j) = (RES_E(i, j + 1) - RES_E(i, j)) / RES_E(i, 1)$$

Разностная характеристика определяется по наименьшему значению. В дальнейшем этот факт будет использоваться в анализе общих характеристик.

3.6. Стабильность сети

В рамках данной характеристики будем рассматривать стабильность системы как комплекса компонентов. Под *стабильностью сети* будем подразумевать предсказуемую, надежную и непрерывную передачу данных с минимальными отклонениями в качестве и джиттером сети.

В рамках тестирования на вход каждому сервису подаётся нагрузка вектор $L_i = (L_{get}, L_{post}, L_{put}, L_{patch}, L_{delete})$, каждые t/r мс считывается матрица задержки $M_{сеть}$. Затем для каждой такой матрицы вычислим разностную матрицу, отображающую насколько нагрузка влияет на задержку между подами:

$$dM_{сеть} = M_{сеть} - M_{стат.сеть}$$

в ходе измерений получим тензор 3-го порядка матрицы задержки по времени t при заданной нагрузке L_i размерности $N*N*r$.

По строкам наблюдаем матрицы размерности $N*r$ которые отражают разницу, которая накладывается нагрузка L_i во времени t . Ожидается, что дисперсия по этим строкам окажется небольшой, в связи со стабильностью нагрузки L_i .

Рассчитаем дисперсию матриц задержек во времени t при заданной нагрузке L_i , тем самым получим взаимосвязь вида $f(L_{get}, L_{post}, L_{put}, L_{patch}, L_{delete}) = DISPNET$. Для установления явного биективного отношения проведем нормализацию:

Пусть $w1, w2, w3, w4, w5$ – некоторые веса, тогда

$$w1*L_{get} + w2*L_{post} + w3*L_{put} + w4*L_{patch} + w5*L_{delete} = DISPNET$$

вычислив веса w_1, w_2, w_3, w_4, w_5 – путём явной подстановки нагрузочных параметров. Таким образом, получив значения весов можем перейти к функции одной переменной:

$$f(L) = \text{DISPNET}$$

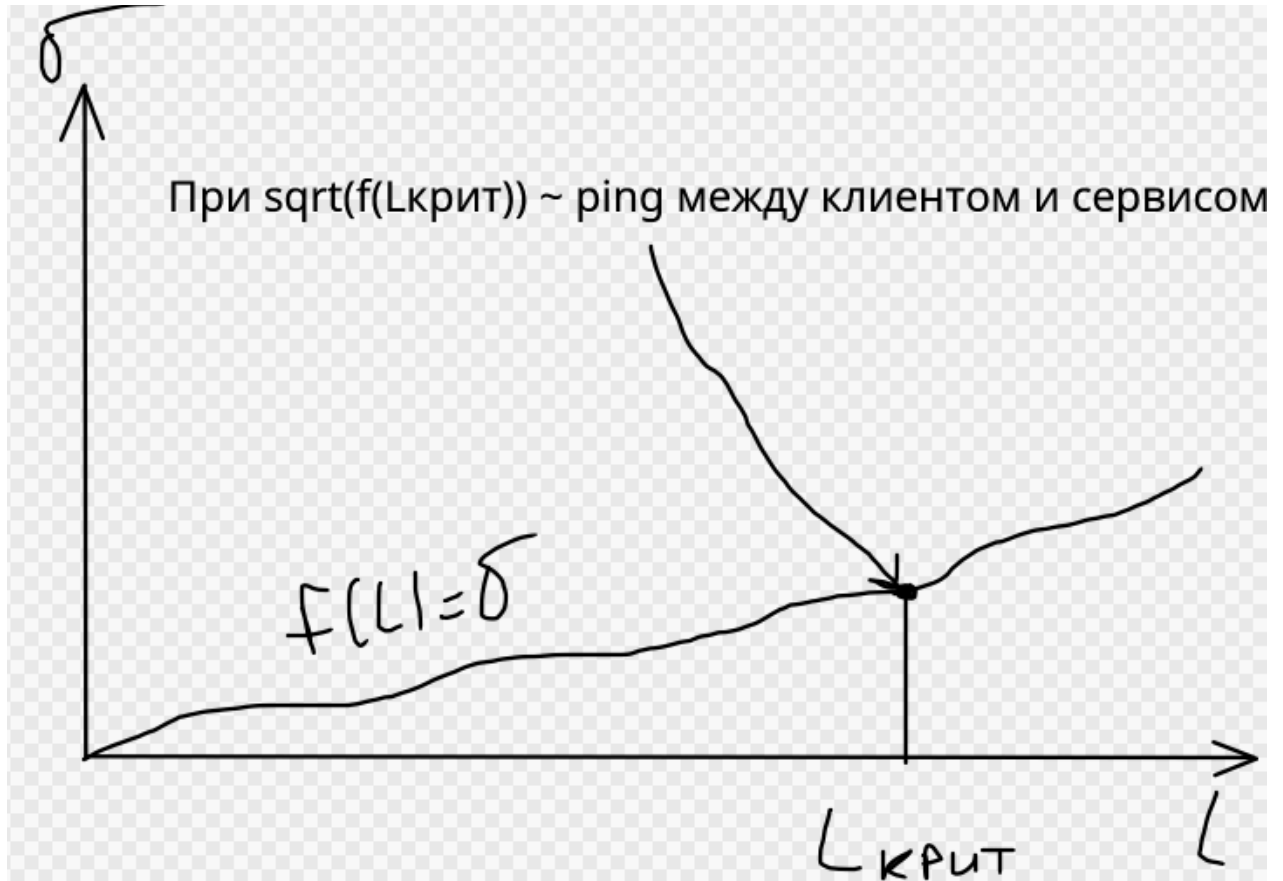


Рисунок 3.8 - Функция $f(L) = \text{DISPNET}$

Получив для заданного количества реплик сервиса данные об задержке между сервисами, мы можем выявить критическую нагрузку пода. Критическая нагрузка на процессор наступает, когда показатели среднего квадратичного отклонения начинают быть сравнимыми с задержкой между клиентом и сервисом.

Составим матрицу размерности $N \times k$, где N - количество микросервисов, k - количество реплик микросервисов, элементы матрицы отображают *критическую нагрузку* обрабатываемое микросервисом N с количеством реплик k . Данная матрица отражает характеристику стабильности сети микросервисного приложения.

Назовём этой матрицу RES_{net} . Выведем из этой матрицы нагрузочные характеристики:

По каждой строке вычислим разностную характеристику:

$$DIFF_{net}(i, j) = (RES_{net}(i, j + 1) - RES_{net}(i, j)) / RES_{net}(i, 1)$$

Разностная характеристика определяется по наименьшему значению. В дальнейшем этот факт будет использоваться в анализе общих характеристик.

3.7. Производительность

В рамках данной характеристики будем рассматривать производительность системы как комплекса компонентов. Она показывает, насколько быстро и плавно приложение реагирует на действия пользователя и выполняет свои задачи.

Определение оценки производительности можно разделить на три подпроцесса:

- Среднее время ответа API.
- Среднее время чтение из базы данных.
- Среднее время записи в базу данных.

В рамках тестирования на вход сервису подаётся нагрузка вектор $L_i = (L_{get}, L_{post}, L_{put}, L_{patch}, L_{delete})$, каждые t/r мс считывается векторы записи в БД, чтения из БД и время отклика – t_{read} , t_{write} и t_{API} . Вычислим для каждого L_i среднее квадратичное отклонение набора значений этих трёх величин. Так как $ср.кв.откл \lll t_{read}, t_{write}$ и t_{API} , то мы можем говорить что для данного сервиса $t_{read}, t_{write}, t_{API} \sim const$.

Проведя для данного микросервиса с данным количеством реплик n тестов: $L_1 \leq \dots \leq L_n$ получим закономерности вида $f(L_{get}, L_{post}, L_{put}, L_{patch}, L_{delete}) = t_{read}, t_{write}, t_{API}$. Для установления явного биективного отношения проведем нормализацию:

Пусть w_1, w_2, w_3, w_4, w_5 – некоторые веса, тогда

$$w_1 * L_{get} + w_2 * L_{post} + w_3 * L_{put} + w_4 * L_{patch} + w_5 * L_{delete} = t_{read}$$

$$w_6 * L_{get} + w_7 * L_{post} + w_8 * L_{put} + w_9 * L_{patch} + w_{10} * L_{delete} = t_{write}$$

$$w_{11} * L_{get} + w_{12} * L_{post} + w_{13} * L_{put} + w_{14} * L_{patch} + w_{15} * L_{delete} = t_{API}$$

вычислив веса $w_1, \dots, w_{15}$ – путём явной подстановки нагрузочных параметров.

Таким образом, получив значения весов можем перейти к функциям одной переменной:

$$f(L) = t_{\text{read}}$$

$$g(L) = t_{\text{write}}$$

$$u(L) = t_{\text{API}}$$

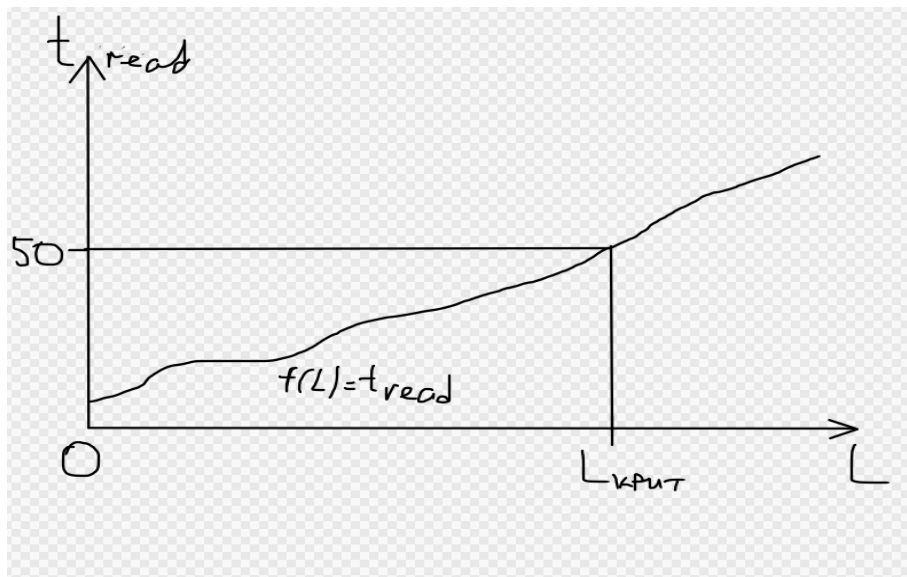


Рисунок 3.9 - Функция $f(L) = f(L) = t_{\text{read}}$

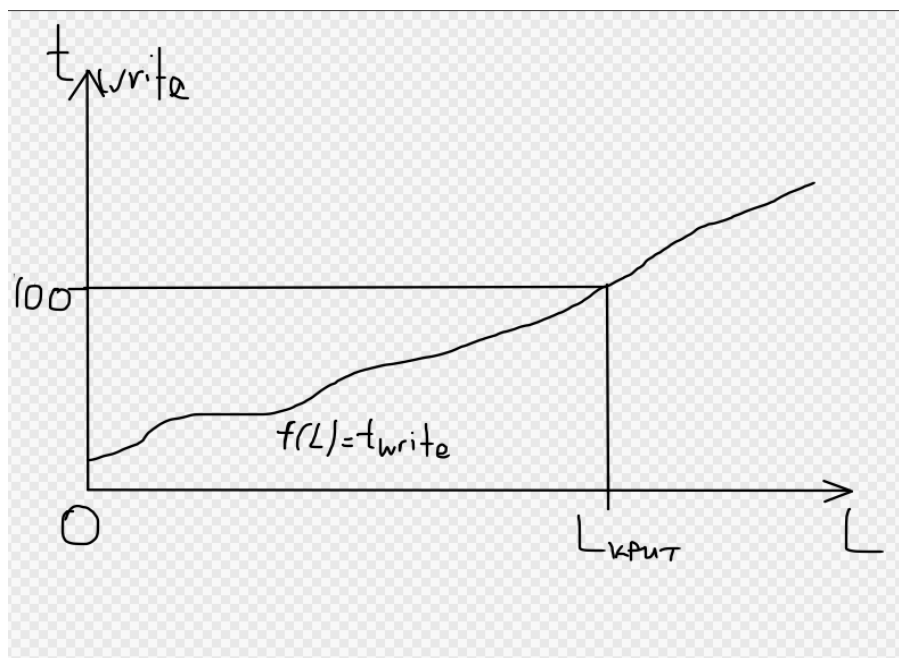


Рисунок 3.10 - Функция $g(L) = t_{\text{write}}$

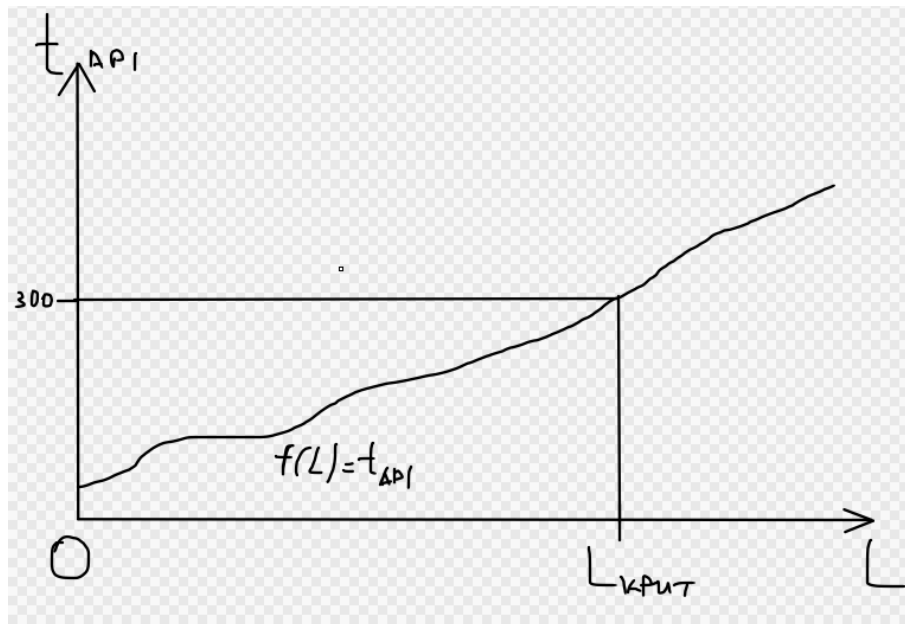


Рисунок 3.11 - Функция $u(L) = t_{API}$

Получив для заданного количества реплик сервиса данные об производительности, мы можем выявить критическую нагрузку пода. Критическая нагрузка наступает, когда наблюдается:

- Время задержки становится более 300мс.
- Время на чтение начинает превышать 50 мс.
- Время на запись начинает превышать 100 мс.

Составим матрицы размерности $N \times k$, где N - количество микросервисов, k - количество реплик микросервисов, элементы матрицы отображают *критическую нагрузку* обрабатываемую микросервисом N с количеством реплик k . Данные матрица отражают характеристику производительность микросервисного приложения.

Рассмотрим анализ одной из этой матрицы. Назовём одну из них RES_{load} (две других назовём RES_{read} , RES_{write}). Выведем из этой матрицы нагрузочные характеристики:

По каждой строке вычислим разностную характеристику:

$$DIFF_{load}(i, j) = (RES_{load}(i, j + 1) - RES_{load}(i, j)) / RES_{load}(i, 1)$$

Разностная характеристика определяется по наименьшему значению. В дальнейшем этот факт будет использоваться в анализе общих характеристик.

3.8. Характеристика брокера сообщений

На вход системе подаётся нагрузка $L = [L_1, \dots, L_N]$, где L_i отображает нагрузку на i -ый микросервис. Воспользовавшись матрицей отношений R . Вычислим:

$$L_{res} = L * R$$

Данная матрица нагрузки отражает, какая нагрузка идёт от какого сервиса. Продолжительность данного тестирования – t секунд. Каждые t/r секунд измеряются следующие метрики:

Пропускная способность (Throughput) - это количество работы, которую система может выполнить за единицу времени. Основные единицы измерения:

- Сообщений в секунду (msg/sec) - количество сообщений;
- Мегабайт в секунду (MB/sec) - объем данных;

Задержка (Latency) - время, которое требуется для выполнения одной операции от начала до конца. Единица измерения – секунды.

Таким образом в рамках постоянной нагрузки L_i получим для каждого сервиса следующие взаимосвязи:

$$F_1(L) = Th_1 /* MB/sec */$$

$$F_2(L) = Th_2 /* messages/sec */$$

$$g(L) = Latency$$



Рисунок 3.12 - Функции $F_1(L)$, $F_2(L)$

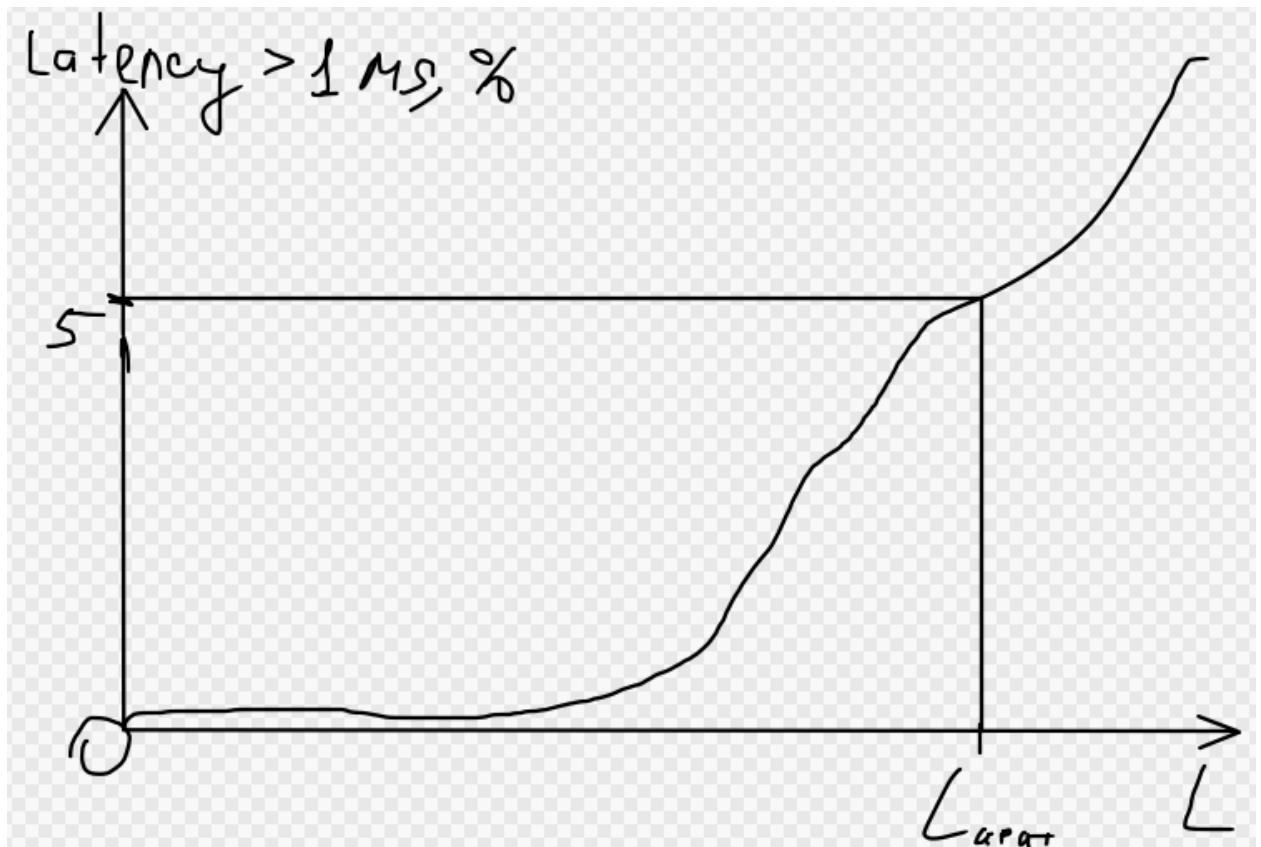


Рисунок 3.13 - Функции $g(L) = \text{Latency}$

где L – результирующая нагрузка на сервис (сумма по строке L_{res}).

Вычислим для каждого L_i среднее квадратичное отклонение набора значений этих трёх величин. Так как $\text{ср.кв.откл} \ll \text{Th}_1, \text{Th}_2$ и Latency , то мы можем говорить что для данного сервиса $\text{Th}_1, \text{Th}_2, \text{Latency} \sim \text{const}$.

Критической нагрузкой $L_{\text{крит}}$ будем называть нагрузку:

- Для пропускной способности: $< 80\%$ от максимальной пропускной способности, которая определена документацией, ресурсами системы.
- Для задержки: более 5% обработанных сообщений было обработано медленнее 1 секунды. ($P95 > 1 \text{ c}$)

Постепенно увеличивая количество подов, получим две матрицы размерности $N \times k$, где N – количество микросервисов, k – количество реплик микросервисов, элементы матрицы отображают *критическую нагрузку* обрабатываемую микросервисом N с количеством реплик k . Данная матрица

отражает характеристику работы брокера сообщений RabbitMQ под действием нагрузки.

Рассмотрим анализ одной из полученных матриц. Назовём одну из них RES_{th} (вторую матрицу назовём $RES_{latency}$). Выведем из этой матрицы нагрузочные характеристики:

По каждой строке вычислим разностную характеристику:

$$DIFF_{th}(i, j) = (RES_{th}(i, j + 1) - RES_{th}(i, j)) / RES_{th}(i, 1)$$

Разностная характеристика определяется по наименьшему значению. В дальнейшем этот факт будет использоваться в анализе общих характеристик.

3.9. Характеристика ошибок и исключений

Частота ошибок, или Error Rate – это соотношение количества запросов, на которые система ответила ошибкой к общему количеству запросов. Под ошибочным запросом будем подразумевать запрос, на который программа ответила некорректно, а также запросы, завершившиеся по истечении времени (time-out error).

В рамках тестирования на вход каждому сервису подаётся нагрузка вектор $L_i = (L_{get}, L_{post}, L_{put}, L_{patch}, L_{delete})$, каждые t/r мс считываются векторы ошибочных запросов, который сразу преобразуется в вектор частоты ошибок $Error_{rate}$. Вычислим для каждого L_i среднее квадратичное отклонение набора ошибок. Так как $ср.кв.откл \lll Error_{rate}$, то мы можем говорить что для данного сервиса $Error \sim const$.

Проведя для данного микросервиса с данным количеством реплик n тестов: $L_1 \leq \dots \leq L_n$ получим закономерности вида $f(L_{get}, L_{post}, L_{put}, L_{patch}, L_{delete}) = Error_{rate}$. Для установления явного биективного отношения проведем нормализацию:

Пусть w_1, w_2, w_3, w_4, w_5 – некоторые веса, тогда

$$w_1 * L_{get} + w_2 * L_{post} + w_3 * L_{put} + w_4 * L_{patch} + w_5 * L_{delete} = Error_{rate}$$

вычислив веса $w_1, \dots, w_5$ – путём явной подстановки нагрузочных параметров.

Таким образом, получив значения весов можем перейти к функциям одной переменной:

$$f(L) = \text{Error}_{\text{rate}}$$

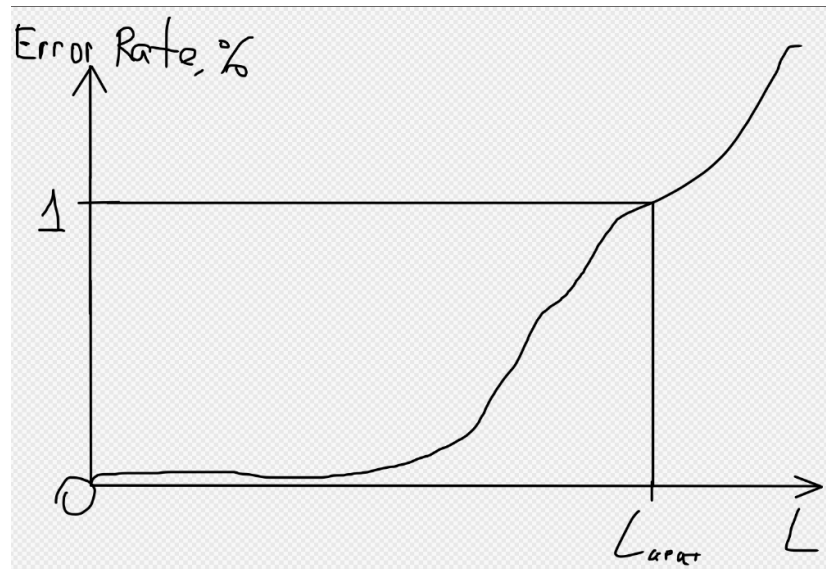


Рисунок 3.14 - Функции $f(L) = \text{Error}_{\text{rate}}$

Получив для заданного количества реплик сервиса данные об ошибках, мы можем выявить критическую нагрузку пода. Критическая нагрузка наступает, когда наблюдается, когда процент ошибок составляет более 1 процента.

Составим матрицы размерности $N \times k$, где N - количество микросервисов, k - количество реплик микросервисов, элементы матрицы отображают *критическую нагрузку* обрабатываемую микросервисом N с количеством реплик k . Данные матрица отражают характеристику производительность микросервисного приложения.

Рассмотрим анализ одной из полученных матриц. Назовём одну из них $\text{RES}_{\text{error}}$. Выведем из этой матрицы нагрузочные характеристики:

По каждой строке вычислим разностную характеристику:

$$\text{DIFF}_{\text{error}}(i, j) = (\text{RES}_{\text{error}}(i, j + 1) - \text{RES}_{\text{error}}(i, j)) / \text{RES}_{\text{error}}(i, 1)$$

Разностная характеристика определяется по наименьшему значению. В дальнейшем этот факт будет использоваться в анализе общих характеристик.

3.10. Общие взаимосвязи

Выше мною был произведён анализ 7 метрик в отрыве друг от друга, но анализ не даёт комплексной оценки «система-сервер-нагрузка».

Для анализа *общего состояния микросервисного приложения* как было сказано ранее будем использовать разностные характеристики.

Разделим все метрики на две группы:

- *Ресурсные*: потребление памяти, потребление ресурса процессора, энергопотребление.
- *Производительные*: стабильность сети, производительность, характеристика обмена в RabbitMQ и Error Rate.

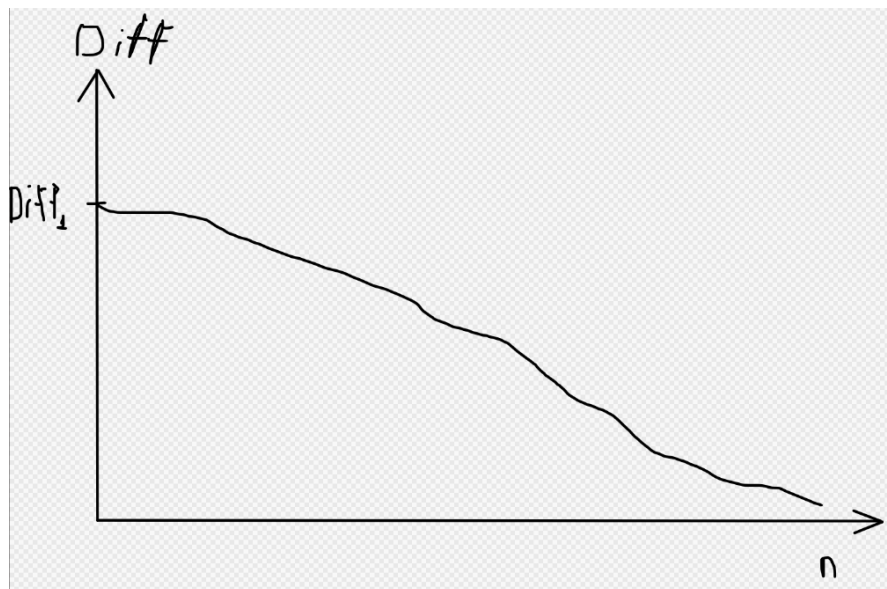


Рисунок 3.15 - График функции вида $f_i(n) = \text{DIFF}$

По каждой метрике создаём матрицу размерности $N \times (k-1)$, элементы которой отражают разностную характеристику. Выводы, которые можно сделать по этим матрицам:

- Высокие значения в строке указывают на то, что сервис положительно реагирует на масштабирование. Он, вероятно, хорошо распараллеливается и является кандидатом для горизонтального масштабирования.

- Можно найти сервис с самой высокой средним значением по строке («звезда» масштабирования) и сервис с самой низким значением («аутсайдер», требующий оптимизации).
- Можно найти группу подов, дающую максимальный эффект для системы в целом («критический ресурс»).
- Матрица подсказывает, является ли система **сбалансированной** (все сервисы примерно одинаково реагируют на масштабирование) или **несбалансированной** (есть явные узкие места и явные лидеры).
- Она помогает выбрать между **равномерным масштабированием**, если матрица сбалансирована, и **приоритетным масштабированием** конкретных сервисов на конкретных типах подов, если матрица несбалансирована.
- Матрица позволяет формализовать задачу оптимизации: *«Имея ограничение на общее количество реплик (например, из-за лимитов стоимости или ресурсов кластера), как распределить их между сервисами, чтобы максимизировать общий прирост производительности?»* - классическая задача на условную оптимизацию и балансировку, которую можно решить, например, жадным алгоритмом: на каждом шаге добавлять одну реплику тому сервису в той группе подов, который дает наибольший текущий прирост.

Введём матрицу эффективности $E(i, j)$ размерности $N \times (k-1)$, элементы которой равны:

$$E(i, j) = P(i, j) / C(i, j)$$

Где $P(i, j)$ и $C(i, j)$ – это матрицы разностных характеристик производительных и ресурсных метрик соответственно. По данной матрице можно сделать выводы по соотношению «производительность-ресурс»:

- Мы можем сделать однозначный вывод по приоритизации масштабирования - всегда масштабируем тот сервис и с того количества реплик, где элемент $E(i, j)$ максимален.

- Сервисы с низкими значениями $E(i, j)$ потребляют много ресурсов относительно скромной пользы. Это кандидаты на глубокую оптимизацию или рефакторинг.
- Можем (*а может и не сможем*) сделать вывод о «диссипации эффективности». *«С ростом j значения $E(i, j)$ для каждого сервиса будут убывать даже быстрее, чем $P(i, j)$.»* Объясняется данный эффект тем, что с ростом числа реплик часто растут накладные расходы на координацию. $C(i, j)$ может оставаться постоянным, в то время как прирост производительности $P(i, j)$ падает. Это приводит к резкому падению эффективности $E(i, j)$.
- Благодаря матрице $E(i, j)$, можем найти экономически целесообразную точку останова. Например, правилом останова будет: *«прекращаем масштабировать сервис i , когда его эффективность становится ниже, чем у следующего лучшего кандидата на масштабирование - сервиса j »*
- Теперь можно объективно сравнивать сервисы между собой, тем самым определять направления рефакторинга.
 - У сервисов-трудяг наблюдаются высокие значения $P(i, j)$ и $E(i, j)$. Они масштабируются очень эффективно.
 - У сервисов-прожор наблюдаются высокие значения $P(i, j)$, но низкие $E(i, j)$. Прожоры дают хороший прирост, но непомерно дорого стоят.
 - У сервисов-кандидатов на оптимизацию наблюдаются низкие значения $P(i, j)$ и $E(i, j)$, следовательно их масштабирование почти бесполезно и неэффективно, необходим рефакторинг.
 - У сервисов с постоянной эффективностью значения $E(i, j)$ почти не падают. Это идеал горизонтального масштабирования.

Во время проведения итерационного тестирования мы вычисляли веса $w_1, \dots, w_5$ в рамках задачи линейной регрессии для различных метрик. Так как

в ходе набора было вычислено достаточно большое количество наборов весов мы можем сделать следующие вывод:

«После ввода системы в эксплуатацию и последующего сбора данных по посещаемости страниц (эндпоинтов). Мы однозначно можем сказать о том, выдержит ли наш сервис нагрузку в час-пик, когда система испытывает наибольшую нагрузку, а также провести масштабирование/демасштабирование системы на основе увеличения/уменьшения популярности страниц(эндпоинтов) у пользователей».

ЗАКЛЮЧЕНИЕ

В рамках курсовой работы было проведено исследование, целью которого является повышение эффективности информационных систем в сфере туризма по таким критериям, как производительность, так и отказоустойчивость.

После анализа предметной области и выяснения минусов существующих решений была сформулирована задача исследования, а также основной функционал разрабатываемой в ходе исследования системы. В результате исследования разработана методика выбора архитектурного паттерна для информационных систем, основанная на многокритериальном анализе специфических требований предметной области, а также были установлены неявные закономерности между требованиями к информационной системе и её реализацией на основе микросервисного паттерна.

Проведён сравнительный анализ архитектурных решений и практик. В результате применения методики удалось повысить производительность информационной системы и снизить общее количество фатальных ошибок на.

Практическая значимость работы заключается в разработке архитектурной модели и методики, которые обеспечивают обоснованный подход к созданию высоконагруженных и отказоустойчивых информационных систем в области туризма. Результаты работы позволяют проектировать системы, способные стабильно функционировать в условиях пиковых нагрузок, связанных с сезонностью и растущим туристским потоком.

В перспективе планируется расширить экспериментальную базу исследования, протестировав архитектурную модель на симуляторах нагрузки, более точно имитирующих реальные сценарии использования туристических платформ, а также адаптировать разработанную методику выбора архитектуры для смежных предметных областей с высокими требованиями к доступности и отказоустойчивости, таких как финансовые технологии.

СПИСОК ЛИТЕРАТУРЫ

1. Официальный сайт Ассоциации туроператоров. [Электронный ресурс]. Режим доступа: <https://www.atorus.ru/article/mirovoy-turizm-vyros-v-pervoy-pолоvine-2025-goda-na-5-64050>.
2. Официальный сайт Министерства экономического развития [Электронный ресурс]. Режим доступа: https://economy.gov.ru/material/directions/np_turizm_i_gostepriimstvo/
3. Официальный сайт Правительства России. [Электронный ресурс]. Режим доступа: <http://government.ru/rugovclassifier/920/about/>
4. Официальный сайт Федеральной службы государственной статистики. [Электронный ресурс]. Режим доступа: <https://rosstat.gov.ru/statistics/turizm>
5. Михайлова А. Издание Коммерсант. [Электронный ресурс]. – 2021. - Режим доступа: <https://www.kommersant.ru/doc/5018381>
6. Портал «Building Microservices» [Электронный ресурс]. Режим доступа: https://samnewman.io/books/building_microservices_2nd_edition/
7. Ричардсон К. «Паттерны разработки и рефакторинга» - Изд. Питер -2022. – 544 с.
8. Васильев Н.В. Системы в условиях масштабирования в нескольких датацентрах/ Н.В. Васильев//StudNet – 2021. – 7 с.
9. Голиков В.А., Янаева М.В., Рудник Н.Т. Исследование ключевых архитектурных паттернов микросервисов: теоретический анализ их роли в построении отказоустойчивой информационной системы/ В.А. Голиков, М.В. Янаева, Н.Т. Рудник//Вестник науки – 2025. – 12 с.
10. КонсультантПлюс. Письмо Президента Российской Федерации от 30 марта 2002 года № Пр-576 «Основы политики Российской Федерации в области развития науки и технологий на период до 2010 года и дальнейшую перспективу». [Электронный ресурс]. – 2002. – Режим доступа: https://www.consultant.ru/document/Cons_doc_LAW_91403/
11. Поручение Президента Российской Федерации от 21 мая 2006 года № Пр-842 «О корректировке приоритетных направлений развития науки,

технологий и техники в Российской Федерации». [Электронный ресурс]. – 2006. – Режим доступа: <https://web.archive.org/web/20110609222613/http://mon.gov.ru/dok/ukaz/nti/4407/>

12. Официальный сайт Президента России. Указ Президента Российской Федерации от 07.07.2011 г. № 899 «Об утверждении приоритетных направлений развития науки, технологий и техники в Российской Федерации и перечня критических технологий Российской Федерации» [Электронный ресурс]. – 2011. – Режим доступа: <http://kremlin.ru/acts/bank/33514>

13. Лобова А.И., Вершинин Е.В., Фёдоров В.О. Обзор DDOS-атак на IoT устройства / А.И. Лобова, Е.В. Вершинин, В.О. Фёдоров //Нацбезопасность – 2022. – 6 с.

14. PROSELYTE Software Engineering. «Эволюция архитектуры программного обеспечения: монолиты, микросервисы, модульные монолиты и будущее» [Электронный ресурс]. - 2025. - Режим доступа: <https://proselyte.net/architecture-evolution/#history>

15. ISO/IEC/IEEE 42010:2011 / ГОСТ Р 57100-2016: «Системная и программная инженерия. Описание архитектуры». - М. : Стандартинформ, 2019 - 36 с.

16. ISO/IEC 25010:2011 / ГОСТ Р ИСО/МЭК 25010—2015: «Системная и программная инженерия. Требования и оценка качества систем и программного обеспечения. Модели качества систем и программного продукта» - М. : Стандартинформ, 2015 - 36 с.

17. RFC 8259 – 2017 «Формат обмена данными JSON» [Электронный ресурс] – 2017. – Режим доступа: <https://www.protokols.ru/WP/wp-content/uploads/2017/12/rfc8259.pdf> - 6 с.

18. RFC 9110 – 2022 «Семантика HTTP» [Электронный ресурс] – 2022. – Режим доступа: <https://www.protokols.ru/WP/wp-content/uploads/2022/06/rfc9110.pdf> - 87 с.

19. ISO/IEC 19464:2014 «Information technology — Advanced Message Queuing Protocol (AMQP) v1.0 specification» - 2014. – 14 с.
20. Официальный сайт Docker [Электронный ресурс]. Режим доступа: <https://www.docker.com/>
21. Официальная документация Rkt [Электронный ресурс]. Режим доступа: <https://rocket.readthedocs.io/en/latest/README/>
22. Официальный сайт Kubernetes [Электронный ресурс]. Режим доступа: <https://kubernetes.io/>
23. Официальный сайт Docker Swarm [Электронный ресурс]. Режим доступа: <https://docs.docker.com/engine/swarm/>
24. Официальный сайт Nomad [Электронный ресурс]. Режим доступа: <https://developer.hashicorp.com/nomad>
25. Шуляк А.В. «Сравнительный анализ алгоритмов балансировки нагрузки в среде облачных вычислений»/ А.В. Шуляк //Научный журнал – 2021. – 6 с.
26. Ляшов Е.И. «Ресурсоэффективные алгоритмы балансировки нагрузки в распределённых микросервисных архитектурах»/Е.И. Ляшов //Вестник науки – 2025. – 18 с.
27. Хабр. «Сравнение алгоритмов балансировки нагрузки: Round Robin vs. Least Connections vs. IP Hash» [Электронный ресурс]. – 2023 - Режим доступа: <https://habr.com/ru/companies/otus/articles/770248/>
28. Хабр. «Нагрузочное тестирование без самообмана: как планировать фазы и правильно снимать метрики» [Электронный ресурс]. - 2025. - Режим доступа: <https://habr.com/ru/articles/910760/>
29. Официальная документация Apache JMeter [Электронный ресурс]. Режим доступа: <https://jmeter.apache.org/>
30. GuruSoftware «Comprehensive Guide to Download & Install LoadRunner 12.0 [Электронный ресурс]. - 2024. - Режим доступа: <https://www.gurusoftware.com/comprehensive-guide-to-download-install-loadrunner-12-0/>